

2025

nguyen.a.tu@gmail.com

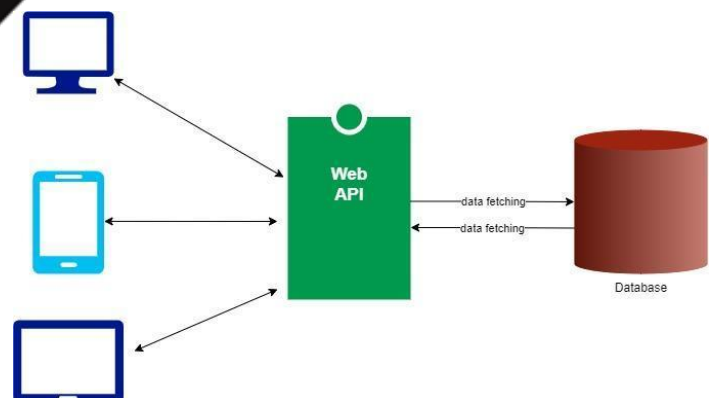
www.facebook.com/nguyenanhtucom

www.youtube.com/@nguyenanhtucom

EXERCISE

Internet of Things

PROGRAMMING



MỤC LỤC

Setup Development Environment	1
Example 0.01	1
IoT Broker	1
Example 1.01	1
Example 1.02	5
Example 1.03	13
IoT Device	22
Example 2.01	22
Example 2.02	25
Example 2.03	28
Example 2.04	31
Example 2.05	34
Example 2.06	38
Example 2.10	42
Example 2.11	48
Example 2.12	55
Example 2.13	62

Setup Development Environment

Example 0.01

Mục tiêu: Thiết lập môi trường lập trình Internet of Things và tạo project

Yêu cầu:

- ✓ Cài đặt JDK và Visual Studio Code
- ✓ Tạo project có tên project là **example01**

Hướng dẫn:

Bước 1: Cài đặt JDK và Visual Studio Code

JDK Development Kit 17.0.7 downloads

JDK 17 binaries are free to use in production and free to redistribute, at no cost, under the Oracle No-Fee Terms and Conditions.

JDK 17 will receive updates under these terms, until September 2024, a year after the release of the next LTS.

Linux macOS Windows

Product/file description	File size	Download
x64 Compressed Archive	172.19 MB	https://download.oracle.com/java/17/latest/jdk-17_windows-x64_bin.zip (sha256)
x64 Installer	153.28 MB	https://download.oracle.com/java/17/latest/jdk-17_windows-x64_bin.exe (sha256)
x64 MSI Installer	152.07 MB	https://download.oracle.com/java/17/latest/jdk-17_windows-x64_bin.msi (sha256)

Bước 2: Để cài đặt Extension Java cho Visual Studio Code bạn hãy thực hiện Mở Extensions (Ctrl+Shift+X), tìm kiếm **java**



Extension Pack for Java

v0.25.12

Preview

Microsoft [microsoft.com](https://www.microsoft.com) | 20,296,269 | ★★★★★ (62)

Popular extensions for Java development that provides Java IntelliSense, debugging, testing, Maven/Gradle support, project management and more

Disable

Uninstall

Switch to Pre-Release Version



This extension is enabled globally.

Bước 3: Để cài đặt Extension Spring Boot cho Visual Studio Code bạn hãy thực hiện Mở Extensions (Ctrl+Shift+X), tìm kiếm **Spring Boot Extension Pack**



Spring Boot Extension Pack

v0.2.1

VMware [vmware.com](https://www.vmware.com) | 1,643,692 | ★★★★★ (15)

A collection of extensions for developing Spring Boot applications

Installing



Bước 4: Sử dụng Visual Studio Code tạo project **Maven** với thông tin như sau:

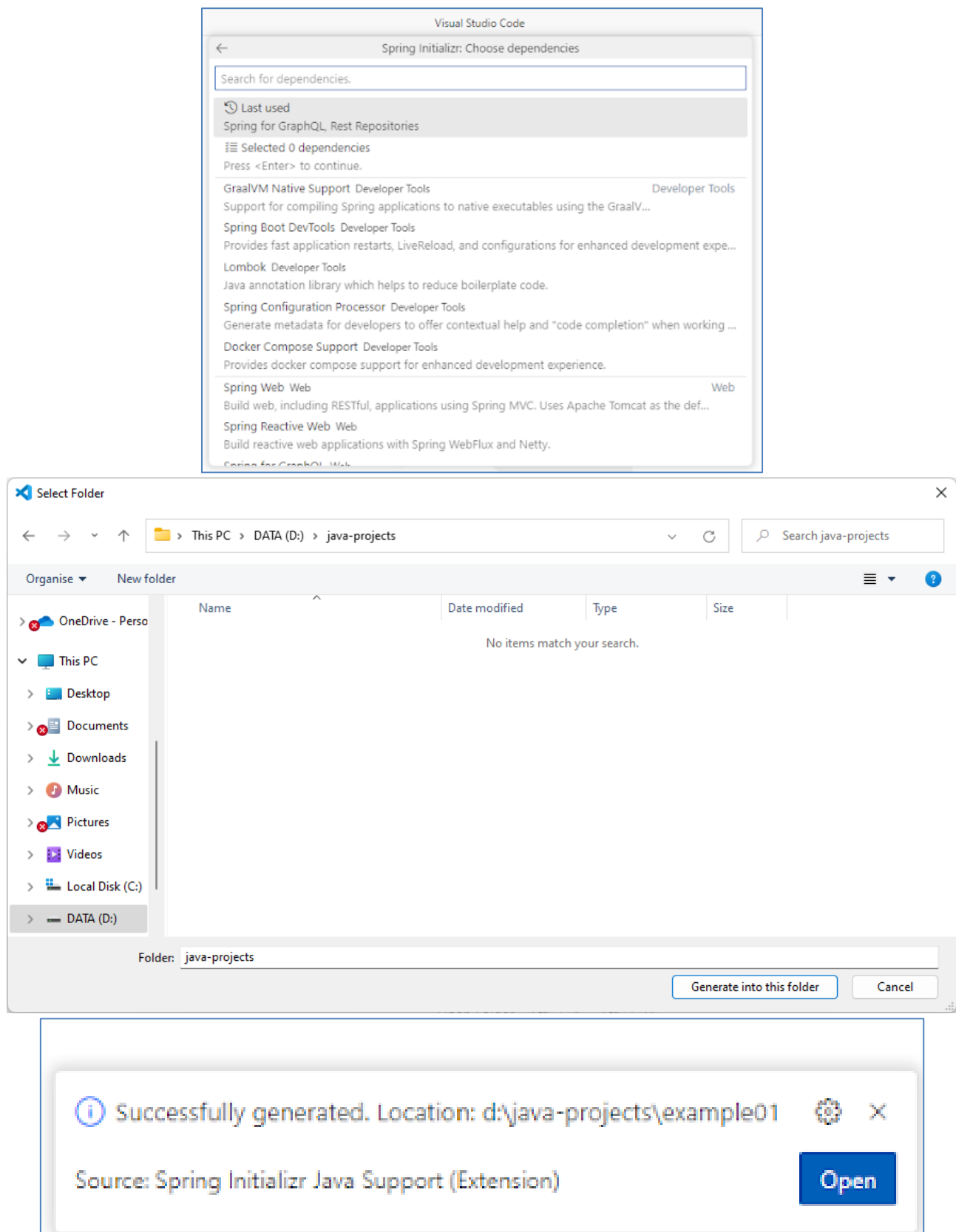
- ✓ Spring Boot version: **3.x.x**
- ✓ Language: **Java**
- ✓ Group Id: **com.nguyenanhtu**
- ✓ Artifact Id: **example01**
- ✓ Packaging type: **Jar**
- ✓ Java Version: **17**
- ✓ Dependencies: **Selected 0 dependencies**

✓ Vị trí project: **D:\java-projects**

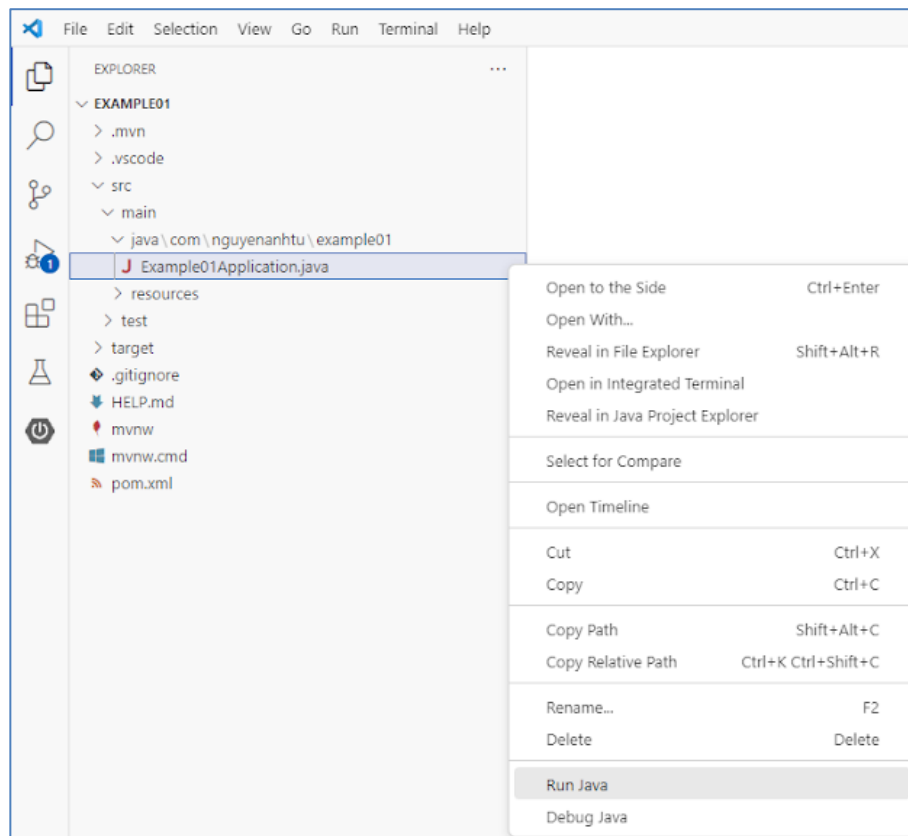
Hãy mở Bảng lệnh (Ctrl+Shift+P) và nhập **Spring Initializr** để bắt đầu tạo dự án **Maven**

The following steps illustrate the Spring Initializr wizard configuration:

- Search:** The command palette shows results for "Spring Initializr", including "Create a Maven Project...", "Add Starters...", and "Create a Gradle Project...".
- Specify Spring Boot version:** A list of versions is shown, with 3.1.1 selected.
- Specify project language:** Options include Java, Kotlin, and Groovy. Java is selected.
- Input Group Id:** The input field contains "com.nguyenanhtu".
- Input Artifact Id:** The input field contains "example01".
- Specify packaging type:** Options include Jar and War. Jar is selected.
- Specify Java version:** A list of versions is shown, with 17 selected.



Bước 5: Click chuột phải vào lớp **Example01Application** sau đó Click vào **Run Java**.



PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```

2023-07-09T11:21:45.752+07:00 INFO 7960 --- [main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 2076 ms
2023-07-09T11:21:46.951+07:00 INFO 7960 --- [main] o.s.b.a.e.web.EndpointLinksResolver : Exposing 1 endpoint(s) beneath base path '/actuator'
2023-07-09T11:21:47.030+07:00 INFO 7960 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port(s): 8080 (http) with context path ''
2023-07-09T11:21:47.046+07:00 INFO 7960 --- [main] c.n.example01.Example01Application : Started Example01Application in 4.152 seconds (process running for 5.033)
2023-07-09T11:21:49.082+07:00 INFO 7960 --- [on(1)-127.0.0.1] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring DispatcherServlet 'dispatcherServlet'
2023-07-09T11:21:49.083+07:00 INFO 7960 --- [on(1)-127.0.0.1] o.s.web.servlet.DispatcherServlet : Initializing Servlet 'dispatcherServlet'
2023-07-09T11:21:49.086+07:00 INFO 7960 --- [on(1)-127.0.0.1] o.s.web.servlet.DispatcherServlet : Completed initialization in 2 ms
  
```

Run: Example01Application

IoT Broker

Example 1.01

Mục tiêu: Quản lý ứng dụng **MQTT-Explorer** kết nối **MQTT Broker** có kiến trúc như sau:



Yêu cầu: Thực hiện các chức năng sau:

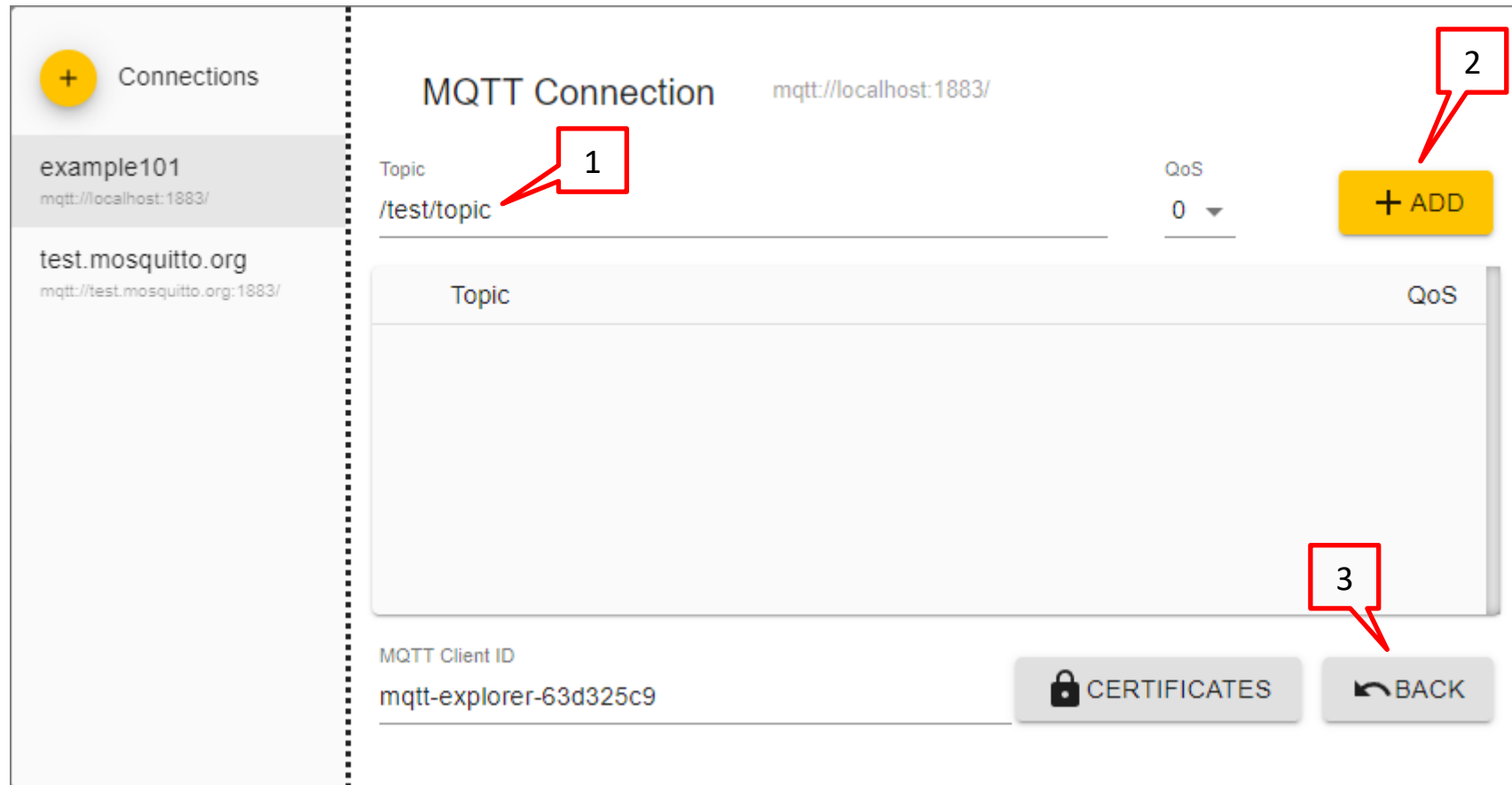
- ✓ Sử dụng **MQTT-Explorer** Subscribe với **Broker** với topic có tên */test/topic*
- ✓ Sử dụng **MQTT-Explorer** Publish nội dung "Hi from the IoT application" đến **Broker** với topic có tên */test/topic*

Hướng dẫn:

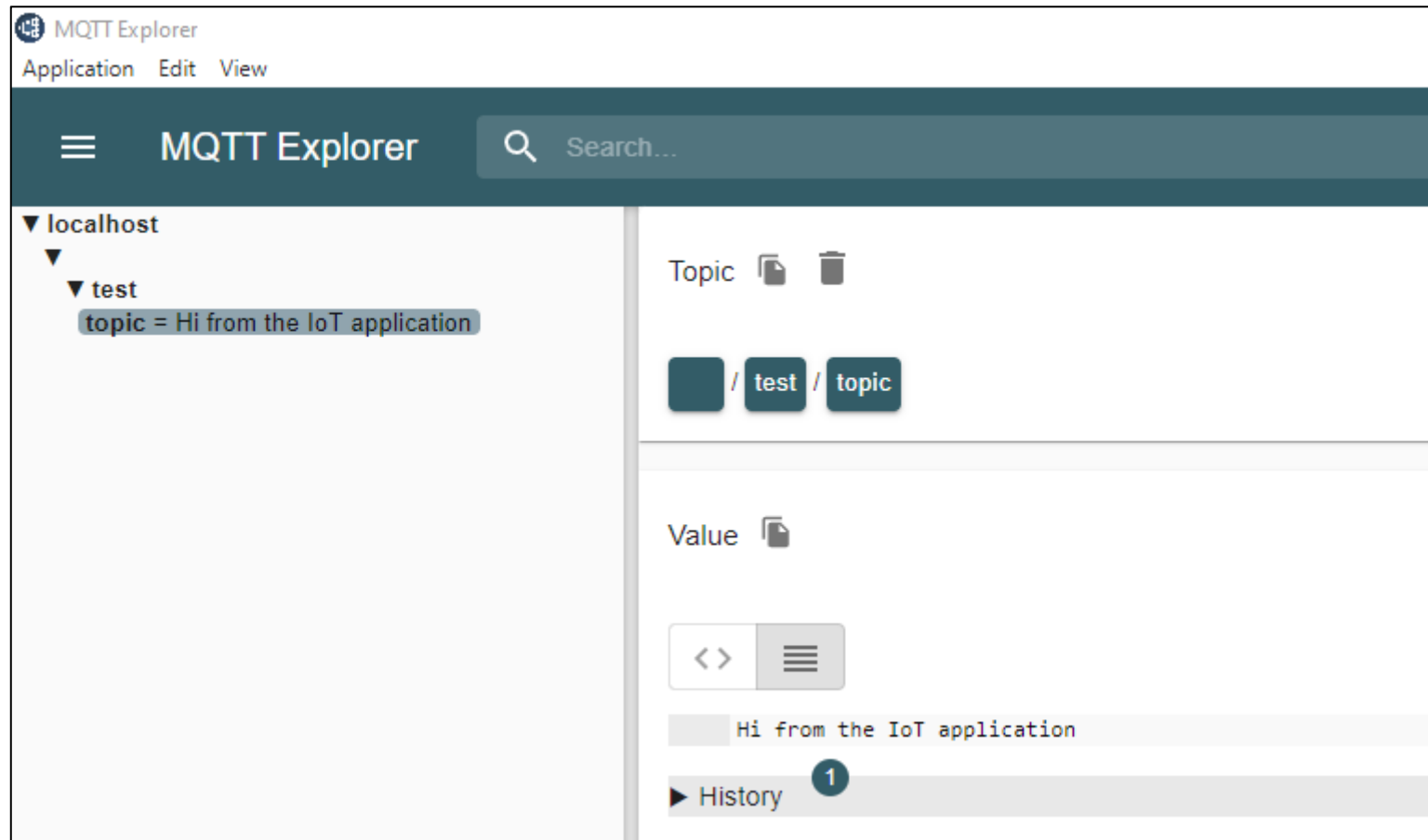
Bước 1: Sử dụng Visual Studio Code, mở Command Palette (Ctrl+Shift+P) và nhập **Java: Create Java Project...** để bắt đầu tạo project với các thông tin như sau:

Bước 1: Sử dụng **MQTT-Explorer** đăng ký topic có tên **/test/topic** với Broker

The screenshot displays the MQTT-Explorer application interface. On the left sidebar, under the 'Connections' section, two connections are listed: 'example101' (mqtt://localhost:1883/) and 'test.mosquitto.org' (mqtt://test.mosquitto.org:1883/). The main area shows the configuration for the selected connection, 'example101'. The title 'MQTT Connection' is followed by the URI 'mqtt://localhost:1883/'. The configuration fields include: 'Name' (example101), 'Validate certificate' (toggle), 'Encryption (tls)' (toggle), 'Protocol' (mqtt://), 'Host' (localhost), 'Port' (1883), 'Username' (empty), and 'Password' (empty). At the bottom, there are four buttons: 'DELETE' (trash icon), 'ADVANCED' (gear icon), 'SAVE' (floppy disk icon), and 'CONNECT' (power icon). Red boxes and arrows highlight the following fields: 1 points to the Name field, 2 points to the Host field, 3 points to the Port field, and 4 points to the Username field.



Bước 4: Sử dụng **MQTT-Explorer** publish tới Broker nội dung **"Hi from the IoT application"** với topic có tên **/test/topic**



Example 1.02

Mục tiêu: Tạo và quản lý ứng dụng **Java** sử dụng **MQTT Paho** kết nối **MQTT Broker** có kiến trúc như sau:



Yêu cầu: Ứng dụng **Java** Thực hiện các chức năng sau:

- ✓ Subscribe với **Broker** với topic có tên */test/topic*
- ✓ Publish nội dung "Hi from the IoT application" đến **Broker** với topic có tên */test/topic*

Hướng dẫn:

Bước 2: Sửa file pom.xml như sau:**pom.xml**

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.nguyenanhtu</groupId>
  <artifactId>example102</artifactId>
  <version>1.0-SNAPSHOT</version>

  <dependencies>
    <dependency>
      <groupId>org.eclipse.paho</groupId>
      <artifactId>org.eclipse.paho.client.mqttv3</artifactId>
      <version>1.2.5</version>
    </dependency>
  </dependencies>

  <properties>
    <maven.compiler.source>17</maven.compiler.source>
    <maven.compiler.target>17</maven.compiler.target>
  </properties>
</project>
```

Bước 4: Tạo Entity **Employee** như sau:**Main.java**

```
package com.nguyenanhtu.example101;

import java.io.IOException;

import org.eclipse.paho.client.mqttv3.IMqttDeliveryToken;
import org.eclipse.paho.client.mqttv3.MqttCallback;
import org.eclipse.paho.client.mqttv3.MqttException;
import org.eclipse.paho.client.mqttv3.MqttMessage;
import org.eclipse.paho.client.mqttv3.MqttConnectOptions;
import org.eclipse.paho.client.mqttv3.MqttClient;

public class Main {
    public static void main(String[] args) {
        String mqttBroker = "tcp://localhost:1883";
        String mqttTopic = "/test/topic";
        String username = "IoTClient";
        String password = "IoTPass";
        String testMsg = "Hi from the IoT application";
        int qos = 1;

        try {
            MqttClient mqttClient = new MqttClient(mqttBroker, "mqttClient");
            MqttConnectOptions mqttOptions = new MqttConnectOptions();
            mqttOptions.setUserName(username);
            mqttOptions.setPassword(password.toCharArray());
            mqttClient.connect(mqttOptions);

            if (mqttClient.isConnected()) {
                mqttClient.setCallback(new MqttCallback() {
                    @Override
```

```
        public void messageArrived(String topic, MqttMessage message) throws Exception {
            System.out.println("Received message: " + new String(message.getPayload()));
        }

        @Override
        public void connectionLost(Throwable cause) {
            System.out.println("Connection is lost: " + cause.getMessage());
        }

        @Override
        public void deliveryComplete(IMqttDeliveryToken token) {
            System.out.println("Message publish is complete: " + token.isComplete());
        }
    });

    /* The client subscribe to a topic */
    mqttClient.subscribe(mqttTopic, qos);
    /* Preparing a message to be published */
    MqttMessage mqttMsg = new MqttMessage(testMsg.getBytes());
    mqttMsg.setQos(qos);
    /* A message is published on the same subscribed topic */
    mqttClient.publish(mqttTopic, mqttMsg);
}

/* Keep the application open, so that the subscribe operation can tested */
System.out.println("Press Enter to disconnect");
System.in.read();
/* Proceed with disconnecting */
mqttClient.disconnect();
mqttClient.close();

} catch (MqttException e) {
    e.printStackTrace();
}
```

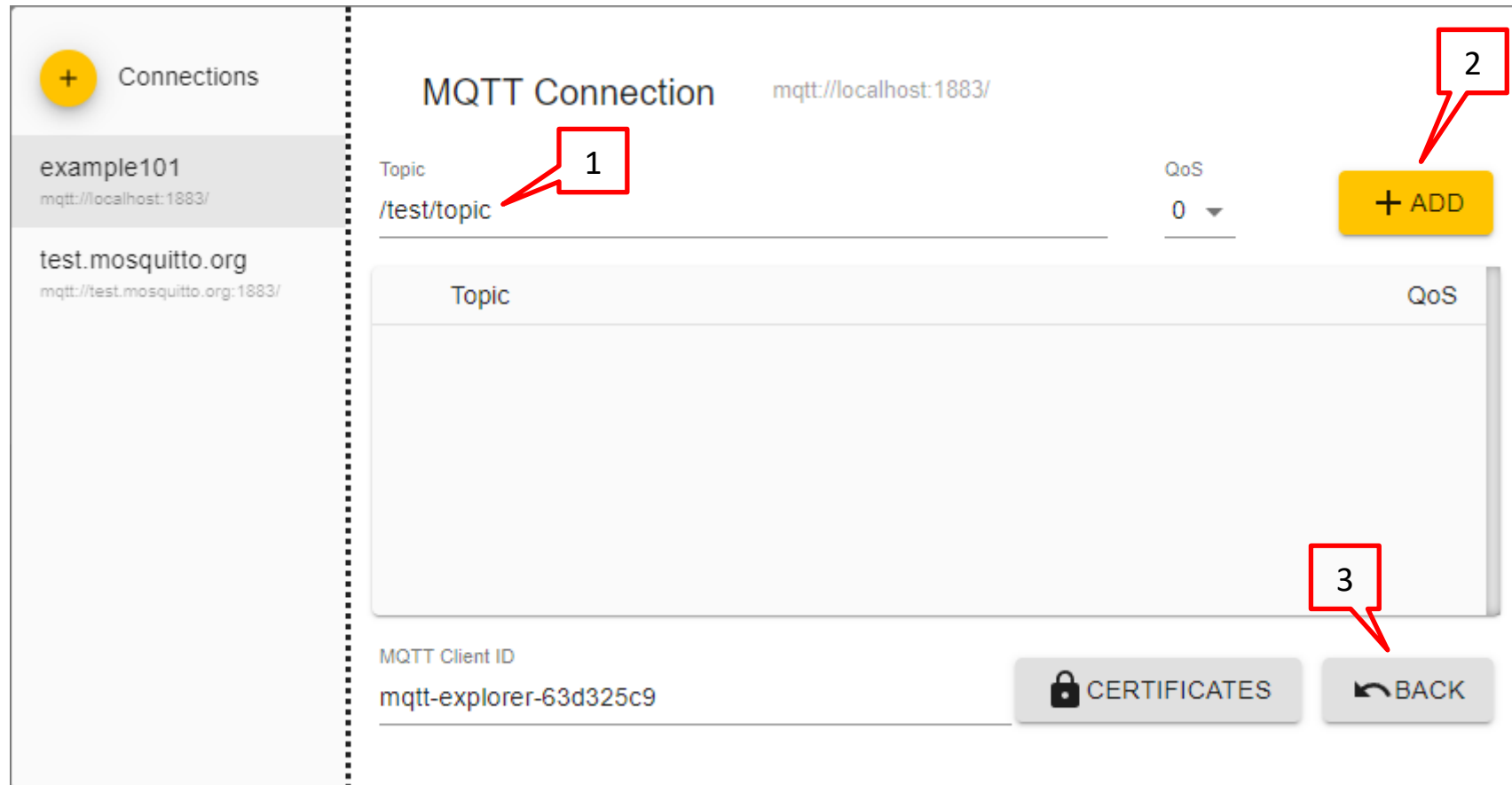
```
} catch (IOException e) {  
    throw new RuntimeException(e);  
}  
}  
}
```

Bước 3: Sử dụng **MQTT-Explorer** đăng ký topic có tên **/test/topic** với Broker

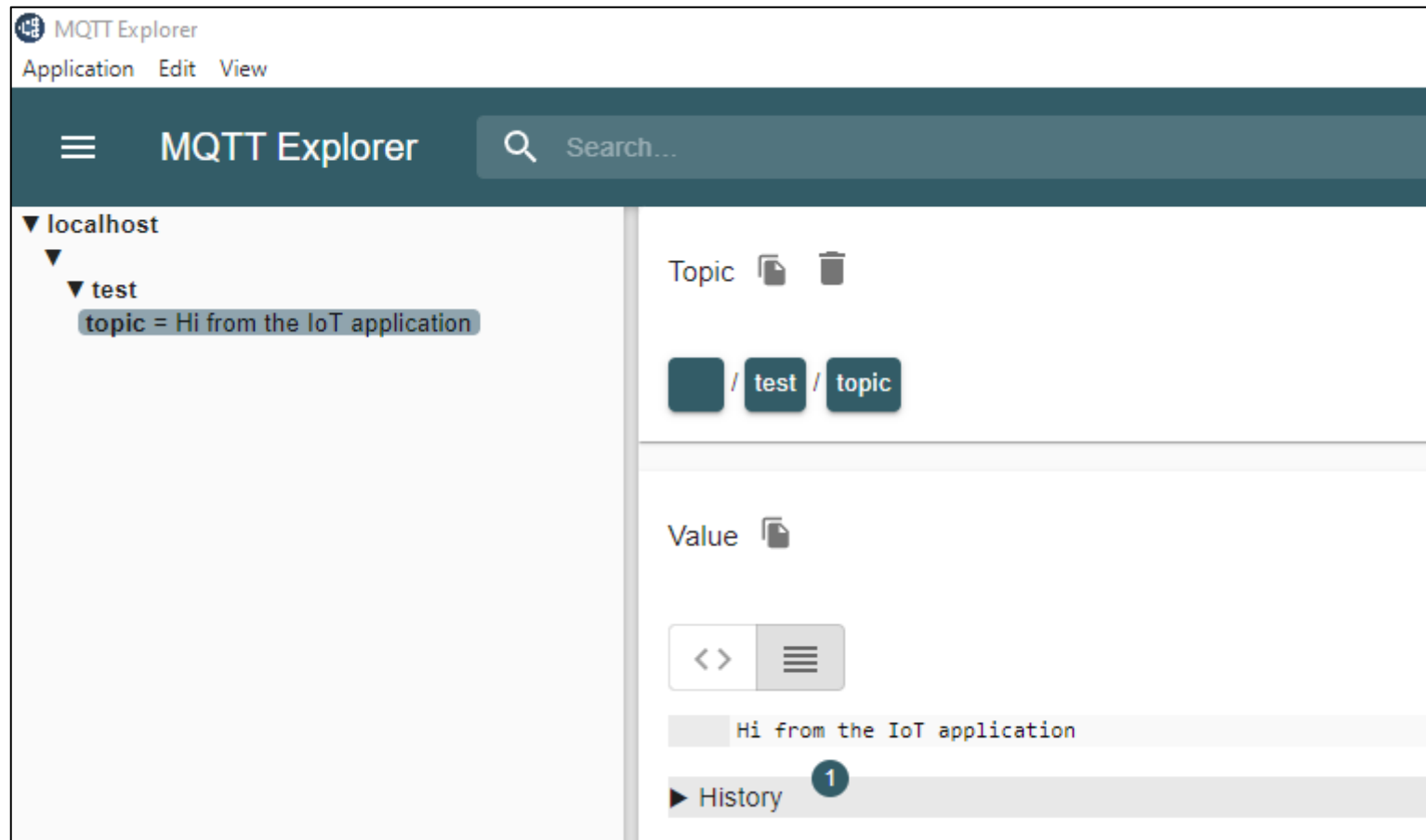
The screenshot displays the MQTT Explorer application interface. On the left, a sidebar shows a list of connections: 'example101' (selected) and 'test.mosquitto.org'. The main panel is titled 'MQTT Connection' and shows the configuration for the selected connection. The configuration includes:

- Name:** 'example101' (labeled with a red box and the number 1).
- Protocol:** 'mqtt://' (labeled with a red box and the number 2).
- Host:** 'localhost' (labeled with a red box and the number 2).
- Port:** '1883' (labeled with a red box and the number 3).
- Username:** (labeled with a red box and the number 4).
- Password:** (with a toggle icon).
- Validate certificate:** (toggle switch).
- Encryption (tls):** (toggle switch).

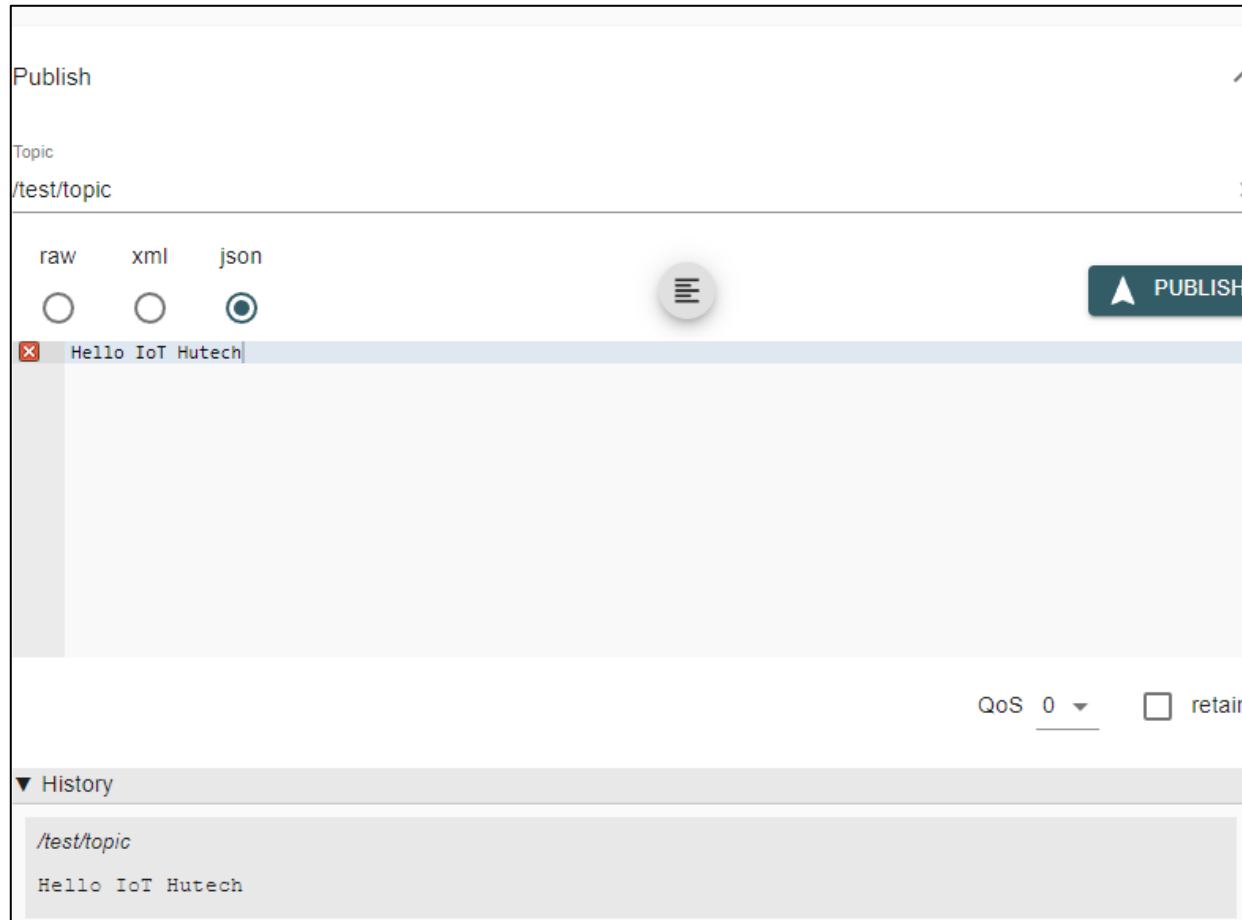
At the bottom, there are four buttons: 'DELETE' (with a trash icon), 'ADVANCED' (with a gear icon), 'SAVE' (yellow button with a disk icon), and 'CONNECT' (blue button with a power icon).



Bước 4: Click chuột phải vào lớp **Main** → **Run Java**, sau đó quan sát ứng dụng **MQTT-Explorer** như sau:



Bước 4: Sử dụng **MQTT-Explorer** publish tới Broker nội dung **Hello IoT Hutech** với topic có tên **/test/topic**

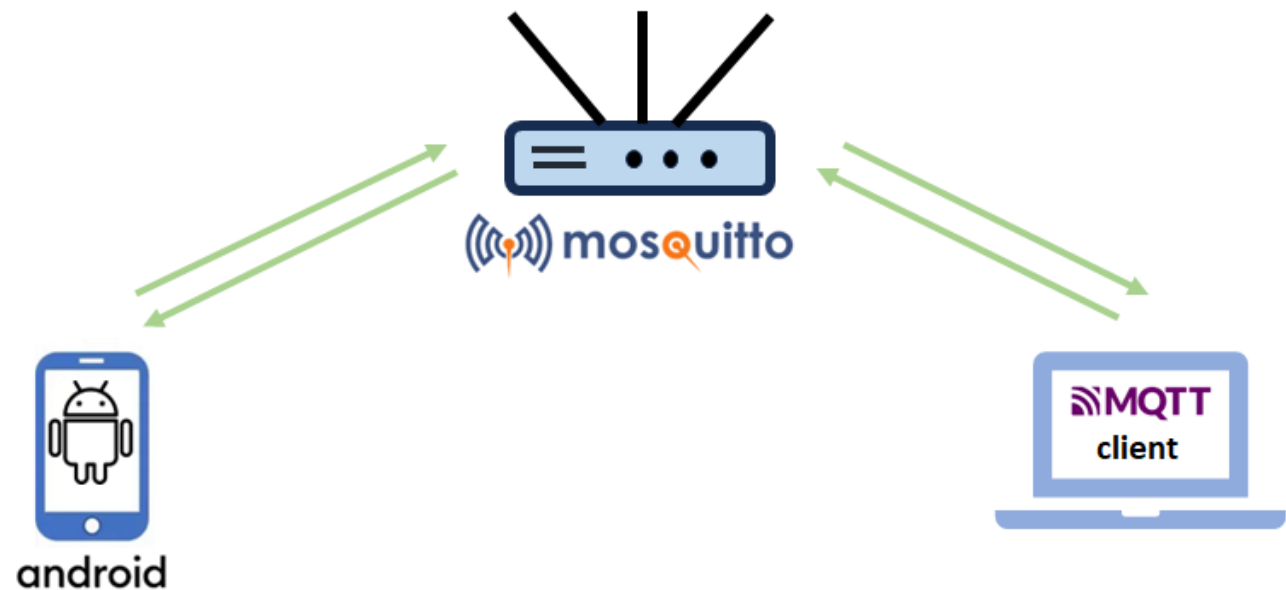


Bước 5: Kiểm tra dữ liệu nhận được trong cửa sổ Terminal của Visual Code

```
Press Enter to disconnect
Received message: Hi from the IoT application
Message publish is complete: true
Received message: Hello IoT Hutech
```

Example 1.03

Mục tiêu: Tạo và quản lý ứng dụng **Android** sử dụng **MQTT** (Eclipse Paho Java MQTT client) kết nối đến **MQTT Broker** có topic tên **/sensor/temp**



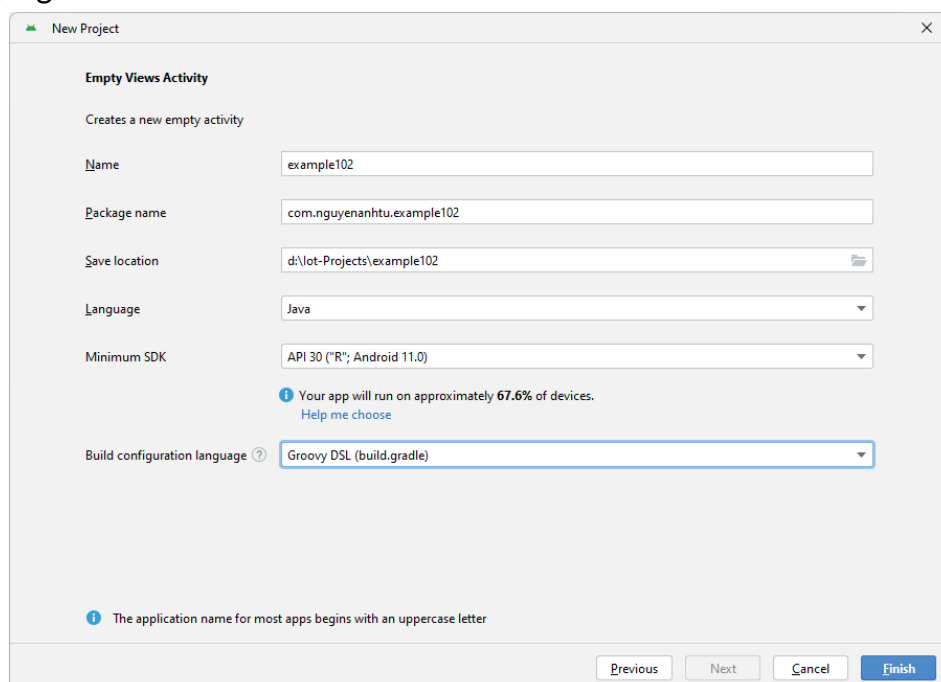
Yêu cầu: Ứng dụng **Android** Thực hiện các chức năng sau:

- ✓ Subscribe với **Broker** với topic có tên **/sensor/temp**

Hướng dẫn:

Bước 1: Sử dụng Android Studio tạo project với các thông tin như sau:

- ✓ Build configuration language: **Groovy DSL(build.gradle)**
- ✓ Vị trí project: **D:\IoT-projects**
- ✓ Artifact Id: **example102**
- ✓ Group Id: **com.nguyenanhtu**
- ✓ Language: **Java**



Bước 2: Thêm vào file **build.gradle** như sau:

build.gradle

```
dependencies {  
    . . .  
    implementation 'org.eclipse.paho:org.eclipse.paho.client.mqttv3:1.2.5'  
    implementation 'org.eclipse.paho:org.eclipse.paho.android.service:1.1.1'  
}
```

Bước 3: Thêm vào file **AndroidManifest.xml** như sau:

AndroidManifest.xml

```
<?xml version="1.0" encoding="utf-8"?>  
<manifest xmlns:android="http://schemas.android.com/apk/res/android"  
    xmlns:tools="http://schemas.android.com/tools">  
  
    <uses-permission android:name="android.permission.INTERNET" />  
    <uses-permission android:name="android.permission.WAKE_LOCK" />  
    <uses-permission android:name="android.permission.ACCESS_NETWORK_STATE" />  
    <uses-permission android:name="android.permission.ACCESS_WIFI_STATE" />  
  
    <application  
        android:allowBackup="true"  
        android:dataExtractionRules="@xml/data_extraction_rules"  
        android:fullBackupContent="@xml/backup_rules"  
        android:icon="@mipmap/ic_launcher"  
        android:label="@string/app_name"  
        android:roundIcon="@mipmap/ic_launcher_round"  
        android:supportRtl="true"  
        android:theme="@style/Theme.Example102"  
        tools:targetApi="31">  
        <activity  
            android:name=".MainActivity"  
            android:exported="true">  
            <intent-filter>  
                <action android:name="android.intent.action.MAIN" />  
                <category android:name="android.intent.category.LAUNCHER" />  
            </intent-filter>  
        </activity>  
    </application>  
</manifest>
```

```
</activity>

    <service android:name="org.eclipse.paho.android.service.MqttService" />

</application>
</manifest>
```

Bước 4: Tạo file **MqttHelper.java** với nội dung như sau:

MqttHelper.java

```
package com.nguyenanhtu.example102;

import android.content.Context;
import android.util.Log;

import org.eclipse.paho.android.service.MqttAndroidClient;
import org.eclipse.paho.client.mqttv3.DisconnectedBufferOptions;
import org.eclipse.paho.client.mqttv3.IMqttActionListener;
import org.eclipse.paho.client.mqttv3.IMqttDeliveryToken;
import org.eclipse.paho.client.mqttv3.IMqttToken;
import org.eclipse.paho.client.mqttv3.MqttCallbackExtended;
import org.eclipse.paho.client.mqttv3.MqttConnectOptions;
import org.eclipse.paho.client.mqttv3.MqttException;
import org.eclipse.paho.client.mqttv3.MqttMessage;

public class MqttHelper {
    public MqttAndroidClient mqttAndroidClient;

    final String serverUri = "tcp://localhost:1883";

    final String clientId = "AndroidClient";
    final String subscriptionTopic = "/sensor/temp";

    final String username = "IoTClient";
    final String password = "IoTClient";

    public MqttHelper(Context context){
```

```
mqttAndroidClient = new MqttAndroidClient(context, serverUri, clientId);
mqttAndroidClient.setCallback(new MqttCallbackExtended() {
    @Override
    public void connectComplete(boolean b, String s) {
        Log.w("mqtt", s);
    }

    @Override
    public void connectionLost(Throwable throwable) {

    }

    @Override
    public void messageArrived(String topic, MqttMessage mqttMessage) throws Exception {
        Log.w("Mqtt", mqttMessage.toString());
    }

    @Override
    public void deliveryComplete(IMqttDeliveryToken iMqttDeliveryToken) {

    }
});
connect();
}

public void setCallback(MqttCallbackExtended callback) {
    mqttAndroidClient.setCallback(callback);
}

private void connect() {
    MqttConnectOptions mqttConnectOptions = new MqttConnectOptions();
    mqttConnectOptions.setAutomaticReconnect(true);
    mqttConnectOptions.setCleanSession(false);
    mqttConnectOptions.setUsername(username);
    mqttConnectOptions.setPassword(password.toCharArray());
}
```

```
try {

    mqttAndroidClient.connect(mqttConnectOptions, null, new IMqttActionListener() {
        @Override
        public void onSuccess(IMqttToken asyncActionToken) {

            DisconnectedBufferOptions disconnectedBufferOptions = new DisconnectedBufferOptions();
            disconnectedBufferOptions.setBufferEnabled(true);
            disconnectedBufferOptions.setBufferSize(100);
            disconnectedBufferOptions.setPersistBuffer(false);
            disconnectedBufferOptions.setDeleteOldestMessages(false);
            mqttAndroidClient.setBufferOpts(disconnectedBufferOptions);
            subscribeToTopic();
        }

        @Override
        public void onFailure(IMqttToken asyncActionToken, Throwable exception) {
            Log.w("Mqtt", "Failed to connect to: " + serverUri + exception.toString());
        }
    });

} catch (MqttException ex) {
    ex.printStackTrace();
}

}

private void subscribeToTopic() {
    try {
        mqttAndroidClient.subscribe(subscriptionTopic, 0, null, new IMqttActionListener() {
            @Override
            public void onSuccess(IMqttToken asyncActionToken) {
                Log.w("Mqtt", "Subscribed!");
            }
        });
    }
}
```

```

        @Override
        public void onFailure(IMqttToken asyncActionToken, Throwable exception) {
            Log.w("Mqtt", "Subscribed fail!");
        }
    });

} catch (MqttException ex) {
    System.err.println("Exception whilst subscribing");
    ex.printStackTrace();
}
}
}

```

Bước 5: Sửa file activity_main.xml như sau:

res/layout/activity_main.xml

```

<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <TextView

        android:id="@+id/dataReceived"

        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>

```

Bước 6: Sửa file MainActivity.java như sau:

MainActivity.java


```
package com.nguyenanhtu.example102;

import androidx.appcompat.app.AppCompatActivity;

import android.os.Bundle;
import android.util.Log;
import android.widget.TextView;

import org.eclipse.paho.client.mqttv3.IMqttDeliveryToken;
import org.eclipse.paho.client.mqttv3.MqttCallbackExtended;
import org.eclipse.paho.client.mqttv3.MqttMessage;

public class MainActivity extends AppCompatActivity {
    MqttHelper mqttHelper;
    TextView dataReceived;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        dataReceived = (TextView) findViewById(R.id.dataReceived);

        startMqtt();
    }

    private void startMqtt() {
        mqttHelper = new MqttHelper(getApplicationContext());
        mqttHelper.setCallback(new MqttCallbackExtended() {
            @Override
            public void connectComplete(boolean b, String s) {

            }

            @Override
            public void connectionLost(Throwable throwable) {

            }

            @Override
            public void messageArrived(String topic, MqttMessage mqttMessage) throws Exception {
```

```
        Log.w("Debug", mqttMessage.toString());
        dataReceived.setText(mqttMessage.toString());
    }

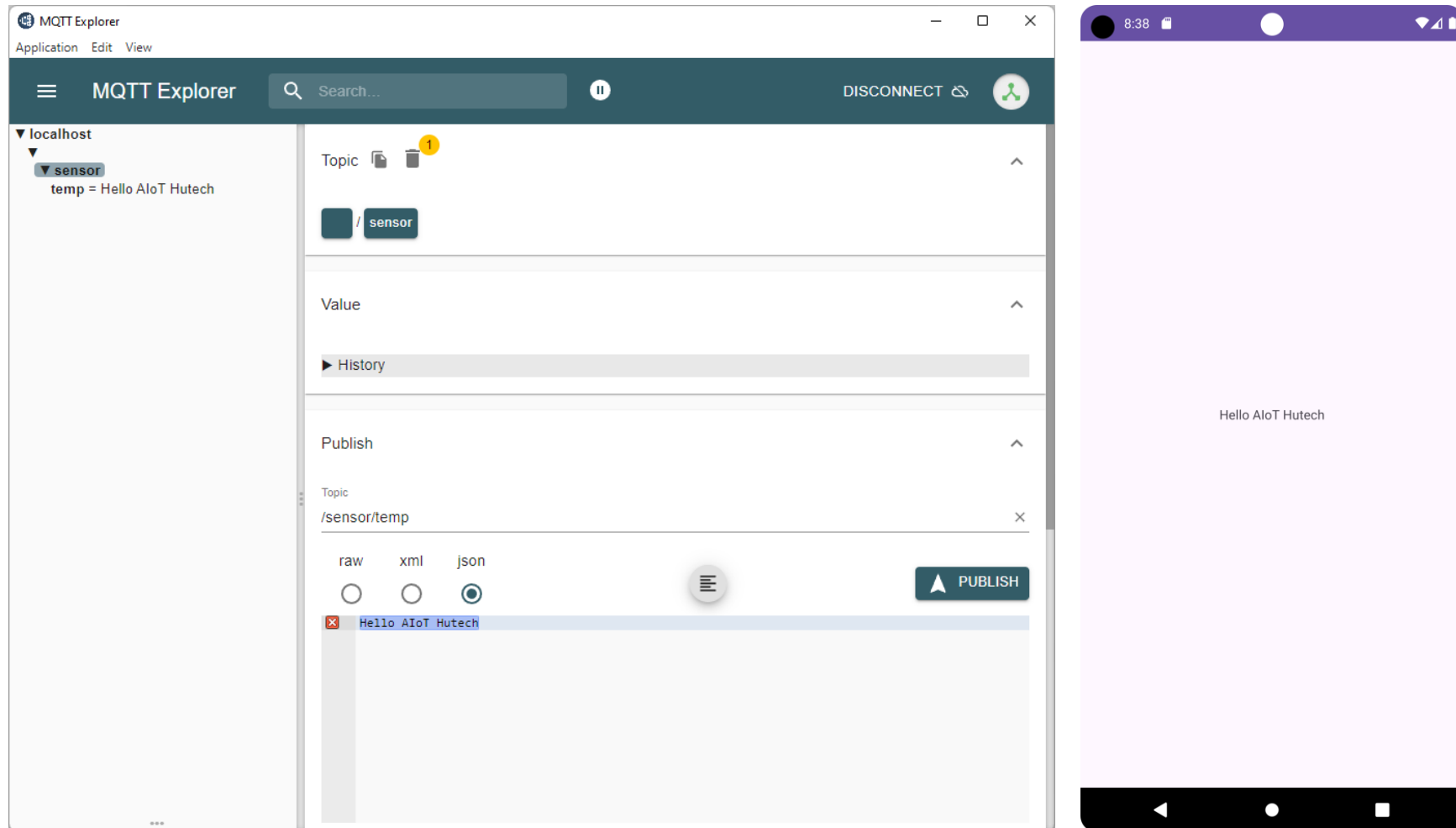
    @Override
    public void deliveryComplete(IMqttDeliveryToken iMqttDeliveryToken) {
    }
}
});
}
```

Bước 7: Sử dụng **MQTT-Explorer** subscribe với Broker một topic có tên **/sensor/temp**

Bước 8: Run App Android



Bước 9: Sử dụng **MQTT-Explorer** publish tới Broker nội dung **Hello AIoT Hutech** với topic có tên **/sensor/temp**



IoT Device

Example 2.01

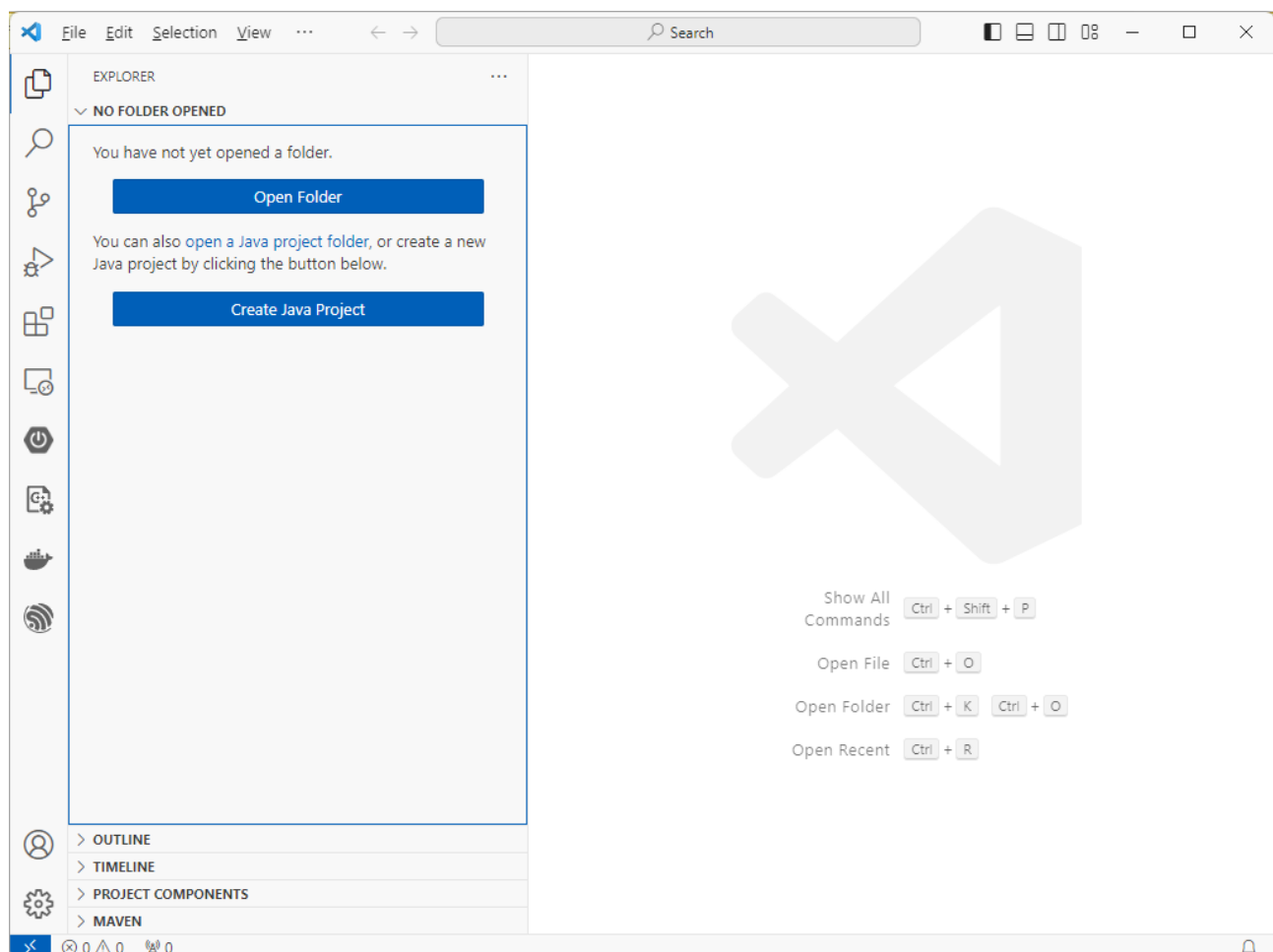
Mục tiêu: Thiết lập môi trường lập trình ESP-IDF (Espressif IoT Development Framework)

Yêu cầu:

- ✓ Cài đặt Visual Studio Code
- ✓ Cài đặt và cấu hình Espressif IDF Extensions

Hướng dẫn:

Bước 1: Download tại <https://code.visualstudio.com>, sau đó tiến hành cài đặt Visual Studio Code



Bước 2: Cài đặt và cấu hình Espressif IDF

- ✓ Để cài đặt Extension Espressif IDF cho Visual Studio Code bạn hãy thực hiện Mở Extensions (Ctrl+Shift+X), tìm kiếm **Espressif IDF**



Espressif IDF

v1.6.5

Espressif Systems espressif.com | 515,985 | ★★★★★ (97)

Develop and debug applications for Espressif ESP32, ESP32-S2 chips with ESP-IDF

[Disable](#) [Uninstall](#) 

This extension is enabled globally.

- ✓ Để cấu hình Extension Espressif IDF cho Visual Studio Code bạn hãy thực hiện Mở Command Palette (Ctrl+Shift+P), chọn **ESP-IDF: Configure ESP-IDF extension**

> ESP-IDF: Configure ESP-IDF extension

ESP-IDF Explorer: Focus on ESP-IDF Docs search results View similar commands 

ESP-IDF: Configure ESP-IDF extension 

ESP-IDF Explorer: Focus on IDF App Tracer View

ESP-IDF: Add Arduino ESP32 as ESP-IDF Component

ESP-IDF: Clear ESP-IDF Search results

 **ESPRESSIF**
ESP-IDF Extension for Visual Studio Code

Welcome.

are installed before choosing the setup mode.

Choose a setup mode.

Select where to save these settings:

Global 

EXPRESS
Fastest option. Choose ESP-IDF, ESP-IDF Tools directory and python executable to create ESP-IDF.

ADVANCED
Configurable option. Choose ESP-IDF, ESP-IDF Tools directory and python executable to create ESP-IDF.
Can choose ESP-IDF Tools download or manually input each existing ESP-IDF tool path.

USE EXISTING SETUP
Select existing ESP-IDF setup saved in the extension or find ESP-IDF in your system.

 **ESPRESSIF**

ESP-IDF Extension for Visual Studio Code

Select download server:

Github ▾

☐ Show all ESP-IDF tags

Select ESP-IDF version:

v5.1.2 (release version) ▾

Enter ESP-IDF container directory

C:\Users\NAT\esp

\esp-idf



Enter ESP-IDF Tools directory (IDF_TOOLS_PATH)

C:\Users\NAT\espressif



Install

Example 2.02

Mục tiêu: Tạo và quản lý ứng dụng chạy trên vi điều khiển **ESP32**:

Yêu cầu: Ứng dụng thực hiện các chức năng sau:

- ✓ In ra Terminal nội dung **Hello World**

Hướng dẫn:

Bước 1: Sử dụng Visual Studio Code, mở Command Palette (Ctrl+Shift+P) và nhập **IDF: New Project**) để bắt đầu tạo project với các thông tin như sau:

- ✓ Project Name: **example202**
- ✓ Enter Project directory: **D:\java-projects**
- ✓ Choose ESP-IDF Board: **ESP32-C3 chip(via ESP USB Bridge)**
- ✓ Choose serial port: **COMx**
- ✓ Choose Template: **template-app**

New Project

Project Name
example201

Enter Project directory
D:\IoT-Projects \example201

Choose ESP-IDF Board
ESP32-C3 chip (via ESP USB Bridge)

Choose serial port
COM1

Add your ESP-IDF Component directory

Choose Template

New Project

ESP-IDF

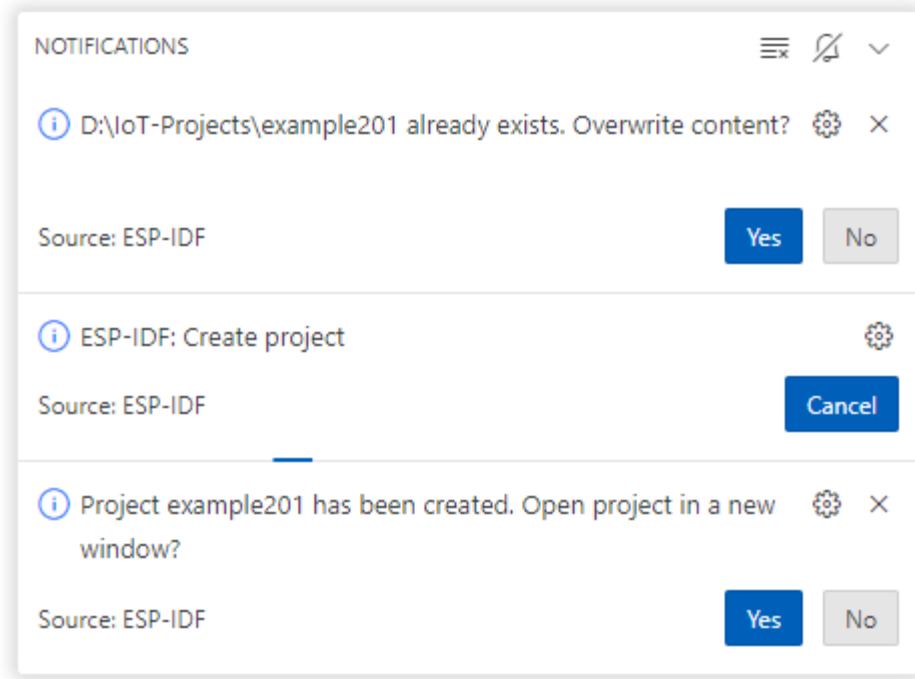
Create project using template hello_world

Supported Targets ESP32 ESP32-C2 ESP32-C3 ESP32-C6 ESP32-H2 ESP32-S2 ESP32-S3

Search Template By Name

get-started
blink
hello_world
sample_project
bluetooth

Hello World Example
Starts a FreeRTOS task to print "Hello World".
(See the README.md file in the upper level 'examples' directory for more information about examples.)



Bước 2: Sửa file **hello_world_main.c** như sau:

hello_world_main.c

```
#include <stdio.h>
#include <inttypes.h>
#include "sdkconfig.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_chip_info.h"
#include "esp_flash.h"

void app_main(void)
```



```
{  
    printf("Hello world!\n");  
    printf("Restarting now.\n");  
    fflush(stdout);  
    esp_restart();  
}
```

Bước 3: Build project và nạp chương trình vào vi điều khiển



Example 2.03

Mục tiêu: Tạo và quản lý ứng dụng sử dụng FreeRTOS chạy trên vi điều khiển **ESP32**:

Yêu cầu: Ứng dụng thực hiện các chức năng sau:

- ✓ In ra Terminal nội dung Hello World

Hướng dẫn:

Bước 1: Sử dụng Visual Studio Code, mở Command Palette (Ctrl+Shift+P) và nhập **IDF: New Project**) để bắt đầu tạo project với các thông tin như sau:

- ✓ Project Name: **example203**
- ✓ Enter Project directory: **D:\java-projects**
- ✓ Choose ESP-IDF Board: **ESP32-C3 chip(via ESP USB Bridge)**
- ✓ Choose serial port: COMx
- ✓ Choose Template: **template-app**

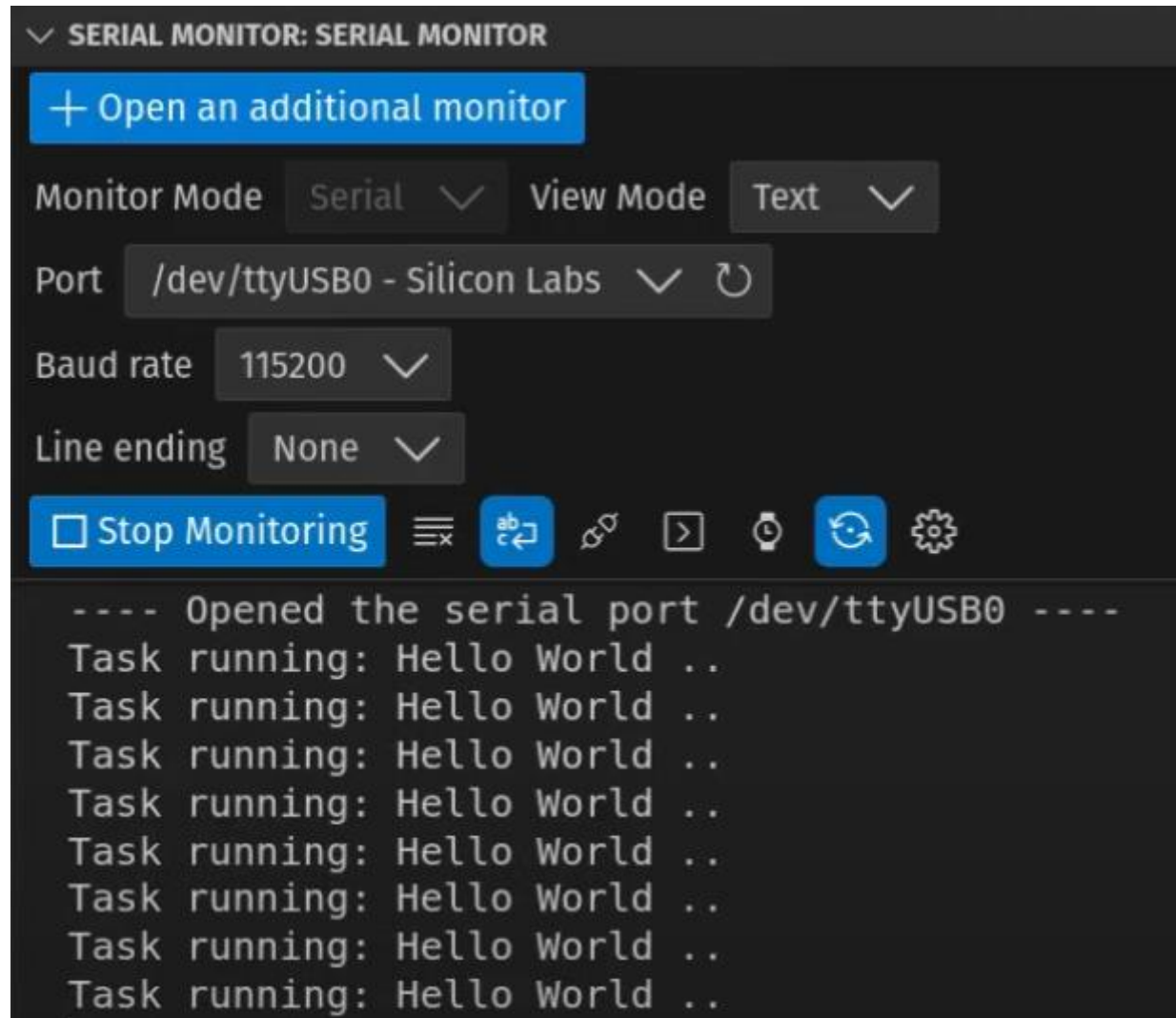
Bước 2: Sửa file `hello_world_main.c` như sau:

hello_world_main.c

```
#include <stdio.h>
#include <inttypes.h>
#include "sdkconfig.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_chip_info.h"
#include "esp_flash.h"
TaskHandle_t HelloWorldTaskHandle = NULL;
void HelloWorld_Task(void *arg)
{
    while (1)
    {
        printf("Task running: Hello World ..\n");
        vTaskDelay(1000 / portTICK_PERIOD_MS);
    }
}
void app_main(void)
{
    xTaskCreate(HelloWorld_Task, "HelloWorld", 4096, NULL, 10, &HelloWorldTaskHandle);
    //xTaskCreatePinnedToCore(HelloWorld_Task, "HelloWorld", 4096, NULL, 10, &HelloWorldTaskHandle, 1); // Run on Core 1
}
```

Bước 3: Build project và nạp chương trình vào vi điều khiển





Example 2.04

Mục tiêu: Tạo và quản lý ứng dụng sử dụng FreeRTOS chạy trên vi điều khiển **ESP32**:

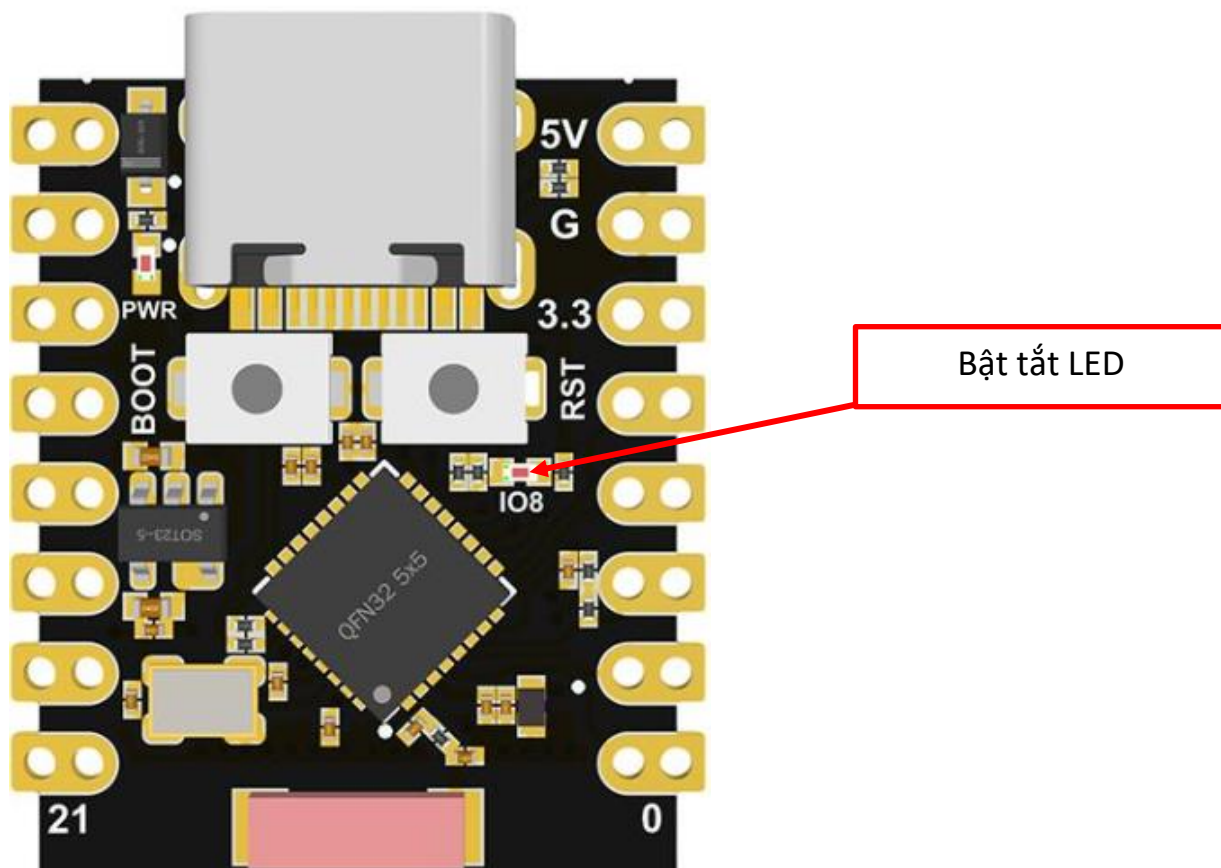
Yêu cầu: Ứng dụng thực hiện các chức năng sau:

- ✓ Thực hiện bật/tắt LED với chu kỳ 1000ms

Hướng dẫn:

Bước 1: Sử dụng Visual Studio Code, mở Command Palette (Ctrl+Shift+P) và nhập **IDF: New Project**) để bắt đầu tạo project với các thông tin như sau:

- ✓ Project Name: **example204**
- ✓ Enter Project directory: **D:\java-projects**
- ✓ Choose ESP-IDF Board: **ESP32-C3 chip(via ESP USB Bridge)**
- ✓ Choose serial port: COMx
- ✓ Choose Template: **template-app**



Bước 2: Sửa file **blink_main.c** như sau:**blink_main.c**

```
#include <stdio.h>
#include <inttypes.h>
#include "sdkconfig.h"
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "esp_chip_info.h"
#include "esp_flash.h"

#include "driver/gpio.h"

#define BLINK_GPIO GPIO_NUM_8

TaskHandle_t BlinkyTaskHandle = NULL;

void Blinky_Task(void *arg)
{
    esp_rom_gpio_pad_select_gpio(BLINK_GPIO);
    gpio_set_direction(BLINK_GPIO, GPIO_MODE_OUTPUT);

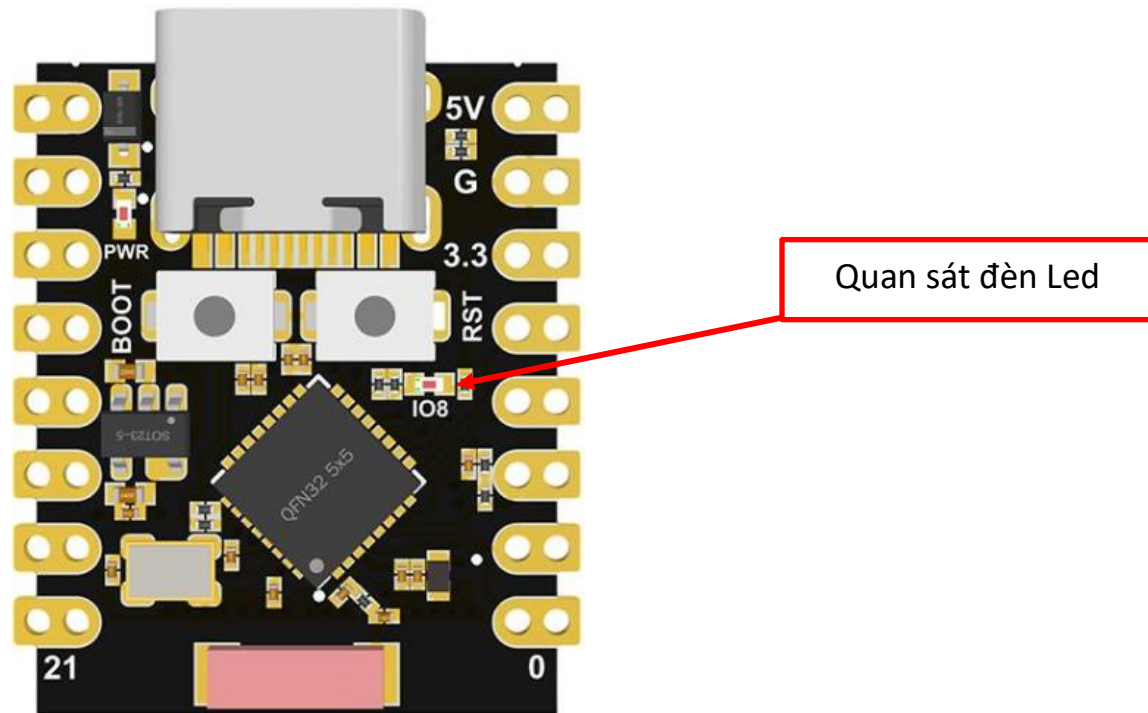
    while (1)
    {
        gpio_set_level(BLINK_GPIO, 1);
        vTaskDelay(1000 / portTICK_PERIOD_MS);
        gpio_set_level(BLINK_GPIO, 0);
        vTaskDelay(1000 / portTICK_PERIOD_MS);
    }
}

void app_main(void)
{

```

```
xTaskCreatePinnedToCore(Blinky_Task, "Blinky", 4096, NULL, 10, &BlinkyTaskHandle, 0); // Core 0  
}
```

Bước 3: Build project và nạp chương trình vào vi điều khiển, sau đó quan sát trạng thái của bóng đèn LED trên board



Example 2.05

Mục tiêu: Tạo và quản lý GIPO. Thực hiện bật tắt đèn Led trên board ESP32-C3

Mục tiêu: Tạo và quản lý ứng dụng sử dụng FreeRTOS chạy trên vi điều khiển **ESP32**:

Yêu cầu: Ứng dụng thực hiện các chức năng sau:

- ✓ Thực hiện bật/tắt LED với chu kỳ 1000ms

Hướng dẫn:

Bước 1: Sử dụng Visual Studio Code, mở Command Palette (Ctrl+Shift+P) và nhập **IDF: New Project**) để bắt đầu tạo project với các thông tin như sau:

- ✓ Project Name: **example204**
- ✓ Enter Project directory: **D:\java-projects**
- ✓ Choose ESP-IDF Board: **ESP32-C3 chip(via ESP USB Bridge)**
- ✓ Choose serial port: COMx
- ✓ Choose Template: **template-app**

Hướng dẫn:

Bước 1: Sử dụng Visual Studio Code tạo Project với thông tin như sau:

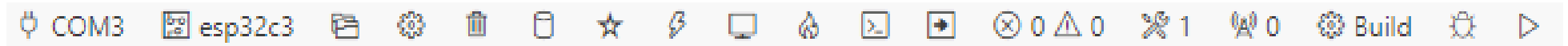
- ✓ Project Name: **Lab02**
- ✓ Choose ESP-IDF Board: **ESP32-C3 chip (via ESP USB Bridge)**
- ✓ Enter Project directory: **D:\IoT-projects**

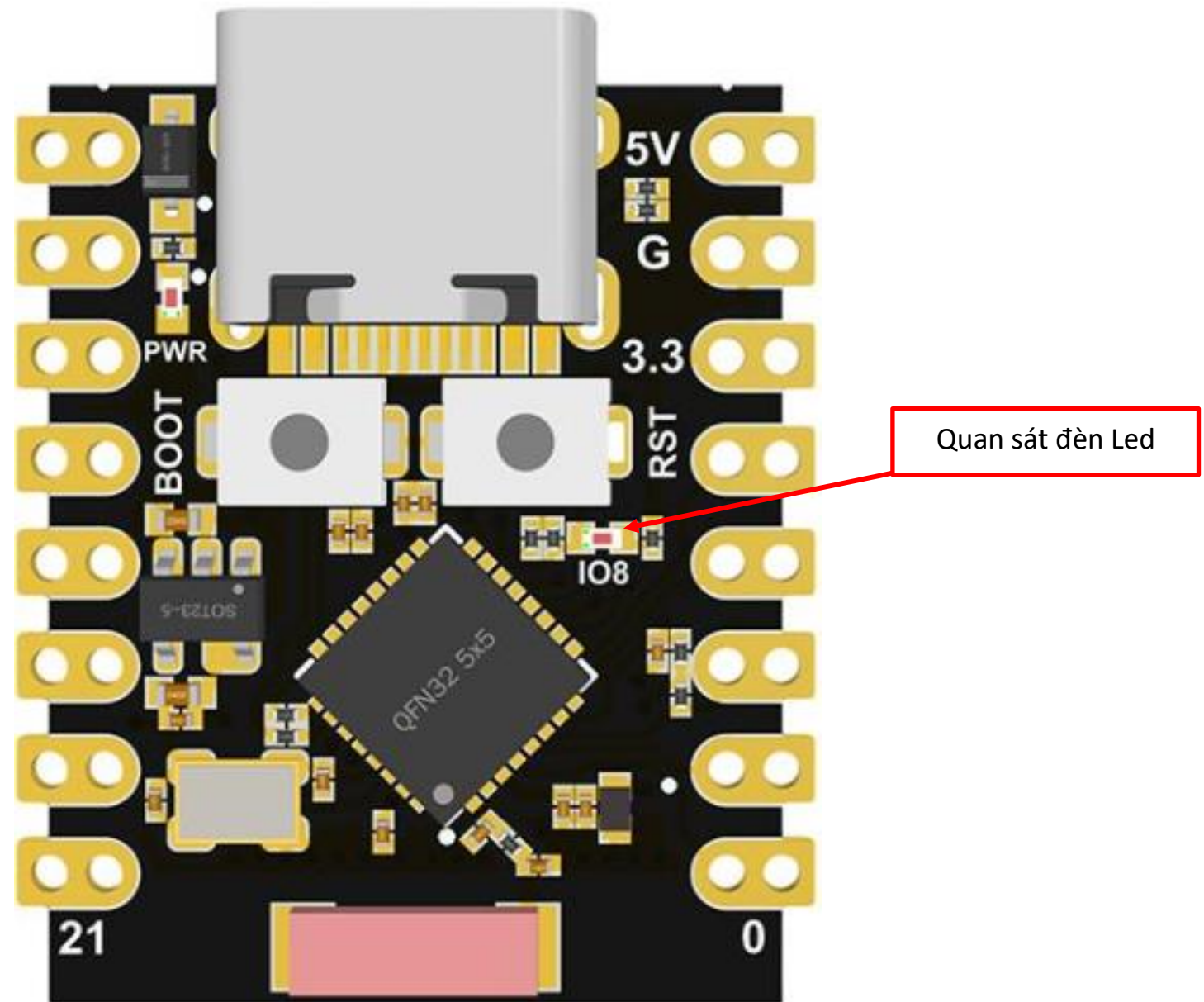
Bước 2: Sửa **main.c** file như sau:

main.c
<pre>#include <stdio.h> #include "driver\gpio.h" #include "freeRTOS\freeRTOS.h" #include "freeRTOS\task.h" #define LED GPIO_NUM_8 void app_main(void) { gpio_set_direction(LED, GPIO_MODE_DEF_OUTPUT); while(1) { gpio_set_level(LED, 1); vTaskDelay(100); gpio_set_level(LED, 0); vTaskDelay(100); } }</pre>

Bước 3: Thực hiện build và flash chương trình lên board ESP32-C3

- ✓ Thực hiện mở Command Palette (Ctrl+Shift+P) sau đó chọn **ESP-IDF: Build your project** và **ESP-IDF: Flash (UART) your project**





Example 2.06

Mục tiêu: Tạo và quản lý ứng dụng sử dụng FreeRTOS chạy trên vi điều khiển **ESP32**:

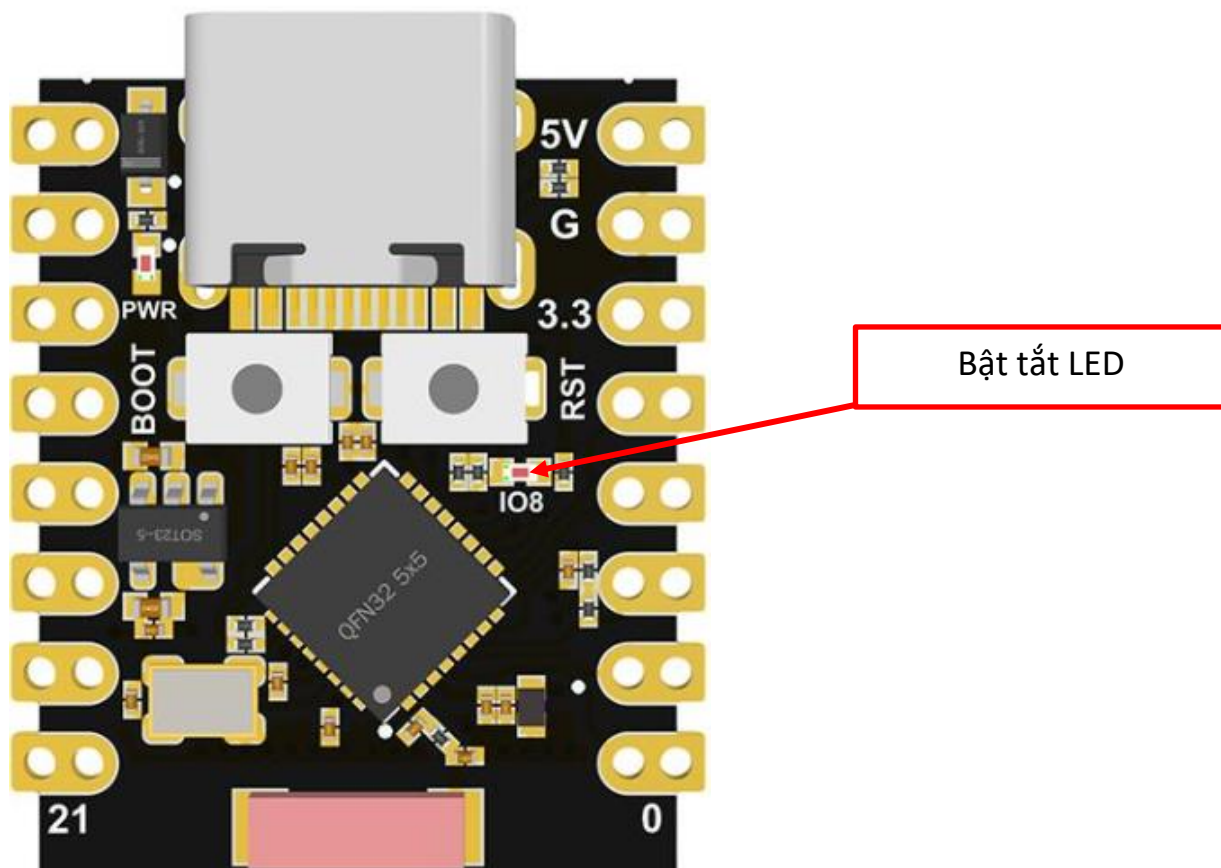
Yêu cầu: Ứng dụng thực hiện các chức năng sau:

- ✓ Thực hiện bật/tắt LED với chu kỳ 1000ms
- ✓ In ra Terminal nội dung Hello World ..

Hướng dẫn:

Bước 1: Sử dụng Visual Studio Code, mở Command Palette (Ctrl+Shift+P) và nhập **IDF: New Project**) để bắt đầu tạo project với các thông tin như sau:

- ✓ Project Name: **example205**
- ✓ Enter Project directory: **D:\java-projects**
- ✓ Choose ESP-IDF Board: **ESP32-C3 chip(via ESP USB Bridge)**
- ✓ Choose serial port: COMx



Bước 2: Sửa file **blink_main.c** như sau:**blink_main.c**

```
#include <stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "driver/gpio.h"
#include "esp_log.h"
#include "sdkconfig.h"

#define BLINK_GPIO GPIO_NUM_8

TaskHandle_t HelloWorldTaskHandle = NULL;
TaskHandle_t BlinkyTaskHandle = NULL;

void HelloWorld_Task(void *arg)
{
    while (1)
    {
        printf("Task running: Hello World ..\n");
        vTaskDelay(1000 / portTICK_PERIOD_MS);
    }
}

void Blinky_Task(void *arg)
{
    esp_rom_gpio_pad_select_gpio(BLINK_GPIO);
    gpio_set_direction(BLINK_GPIO, GPIO_MODE_OUTPUT);

    int count_second = 0;

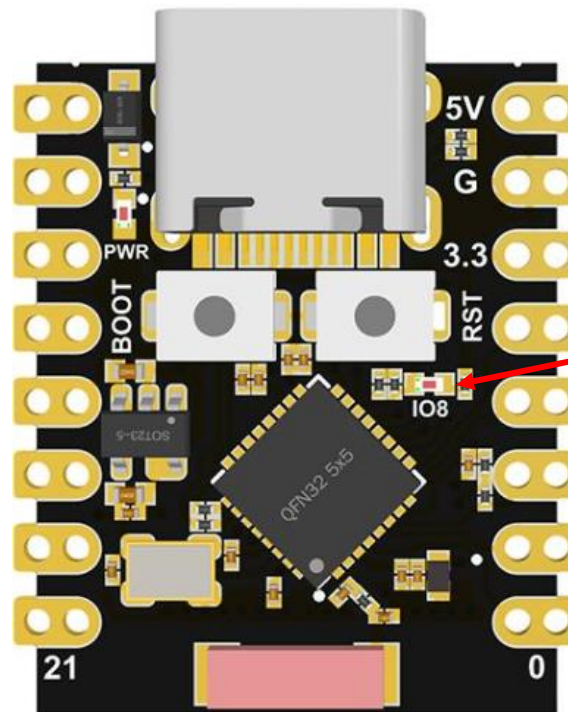
    while (1)
    {
```

```
        count_second += 1;

        switch (count_second)
        {
        case 10:
            vTaskSuspend(HelloWorldTaskHandle);
            printf("HelloWorld task suspended .. \n");
            break;
        case 14:
            vTaskResume(HelloWorldTaskHandle);
            printf("HelloWorld task resumed .. \n");
            break;
        case 20:
            vTaskDelete(HelloWorldTaskHandle);
            printf("HelloWorld task deleted .. \n");
            break;
        default:
            break;
        }
        gpio_set_level(BLINK_GPIO, 1);
        vTaskDelay(1000 / portTICK_PERIOD_MS);
        gpio_set_level(BLINK_GPIO, 0);
        vTaskDelay(1000 / portTICK_PERIOD_MS);
    }
}

void app_main(void)
{
    xTaskCreatePinnedToCore(Blinky_Task, "Blinky", 4096, NULL, 10, &BlinkyTaskHandle, 0); // Core 0
    xTaskCreatePinnedToCore(HelloWorld_Task, "HelloWorld", 4096, NULL, 10, &HelloWorldTaskHandle, 1); // Core 1
}
```

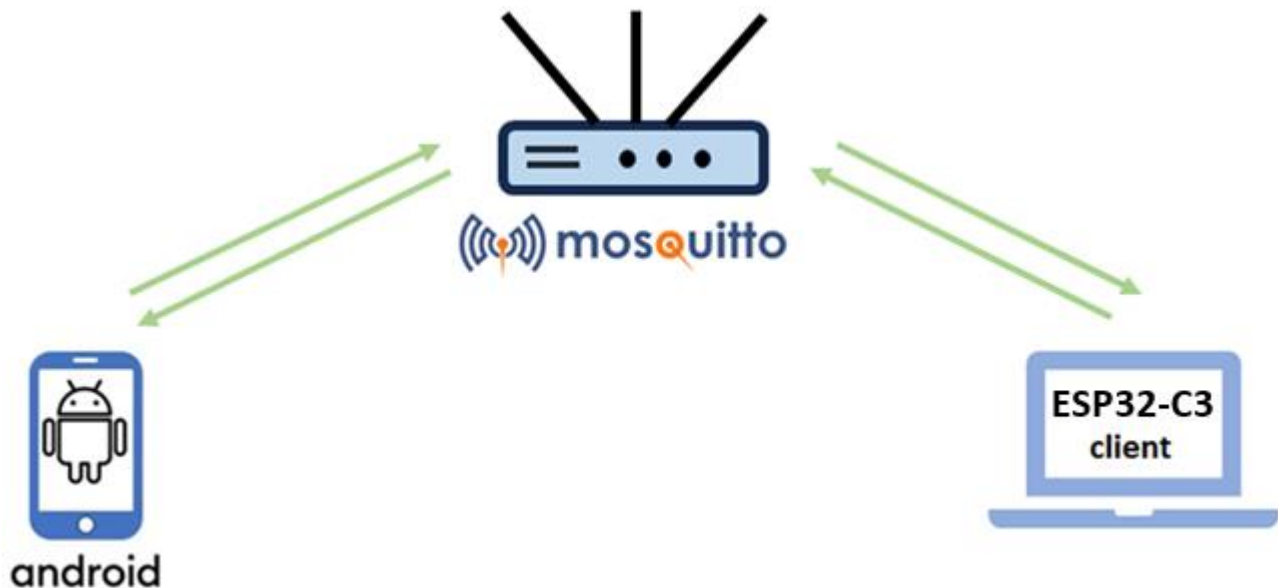
Bước 3: Build project và nạp chương trình vào vi điều khiển, sau đó quan sát trạng thái của bóng đèn LED trên board



Quan sát đèn Led

Example 2.10

Mục tiêu: Tạo và quản lý ứng dụng sử dụng MQTT client chạy trên vi điều khiển **ESP32** (Sử dụng lệnh **idf.py menuconfig** trong ESP-IDF Terminal để cấu hình WIFI) có kiến trúc như sau:



Yêu cầu: Ứng dụng thực hiện các chức năng sau:

- ✓ Sử dụng **ESP32** Subscribe với **Broker** với topic có tên *"/topic/qos0"*
- ✓ Sử dụng **ESP32** Publish nội dung "Hi from the IoT application" đến **Broker** với topic có tên *"/topic/qos1"*

Hướng dẫn:

Bước 1: Cấu hình **MQTT Broker**

<code>... \mosquitto\mosquitto.conf</code>
<pre>listener 1883 allow_anonymous true</pre>

Bước 2: Sử dụng Visual Studio Code, mở Command Palette (Ctrl+Shift+P) và nhập **IDF: New Project** để bắt đầu tạo project với các thông tin như sau:

- ✓ Project Name: **example210**
- ✓ Enter Project directory: **D:\java-projects**
- ✓ Choose ESP-IDF Board: **ESP32-C3 chip(via ESP USB Bridge)**
- ✓ Choose serial port: COMx

Bước 3: Sử dụng lệnh **idf.py menuconfig** trong ESP-IDF Terminal để cấu hình WIFI


```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  ESP-IDF  MEMORY  XRTOS

(Top)
Espressif IoT Development Framework Configuration

Build type --->
Bootloader config --->
Security features --->
Application manager --->
Boot ROM Behavior --->
Serial flasher config --->
Partition Table --->
Example Configuration --->
Example Connection Configuration --->
Compiler options --->
Component config --->
[ ] Make experimental features visible

[Space/Enter] Toggle/enter  [ESC] Leave menu      [S] Save
[O] Load                   [?] Symbol info    [/] Jump to symbol
[F] Toggle show-help mode  [C] Toggle show-name mode [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)

```

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  ESP-IDF  MEMORY  XRTOS

(Top) → Example Connection Configuration
Espressif IoT Development Framework Configuration

[*] connect using WiFi interface
[ ] Get ssid and password from stdin
[*] Provide wifi connect commands
(NAT) WiFi SSID
(0902880088) WiFi Password
(6) Maximum retry
WiFi Scan Method (All Channel) --->
WiFi Scan threshold --->
WiFi Connect AP Sort Method (Signal) --->
[ ] connect using Ethernet interface
[*] Obtain IPv6 address
Preferred IPv6 Type (Local Link Address) --->

[Space/Enter] Toggle/enter  [ESC] Leave menu      [S] Save
[O] Load                   [?] Symbol info    [/] Jump to symbol
[F] Toggle show-help mode  [C] Toggle show-name mode [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)

```

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  ESP-IDF  MEMORY  XRTOS

(Top) → Example Configuration
Espressif IoT Development Framework Configuration

(mqtt://192.168.1.8:1883) Broker URL

[Space/Enter] Toggle/enter  [ESC] Leave menu      [S] Save
[O] Load                   [?] Symbol info    [/] Jump to symbol
[F] Toggle show-help mode  [C] Toggle show-name mode [A] Toggle show-all mode
[Q] Quit (prompts for save) [D] Save minimal config (advanced)

```

Bước 4: Sửa file **app_main.c** như sau:**app_main.c**

```
#include <stdio.h>
#include <stdint.h>
#include <stddef.h>
#include <string.h>
#include "esp_wifi.h"
#include "esp_system.h"
#include "nvs_flash.h"
#include "esp_event.h"
#include "esp_netif.h"
#include "protocol_examples_common.h"

#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/semphr.h"
#include "freertos/queue.h"

#include "lwip/sockets.h"
#include "lwip/dns.h"
#include "lwip/netdb.h"

#include "esp_log.h"
#include "mqtt_client.h"

static const char *TAG = "MQTT_EXAMPLE";

static void mqtt_event_handler(void *handler_args, esp_event_base_t base, int32_t event_id, void *event_data)
```

```
{
    ESP_LOGD(TAG, "Event dispatched from event loop base=%s, event_id=%" PRIi32 "", base, event_id);
    esp_mqtt_event_handle_t event = event_data;
    esp_mqtt_client_handle_t client = event->client;
    int msg_id;
    switch ((esp_mqtt_event_id_t)event_id)
    {
    case MQTT_EVENT_CONNECTED:
        ESP_LOGI(TAG, "MQTT_EVENT_CONNECTED");
        msg_id = esp_mqtt_client_publish(client, "/topic/qos1", "data_3", 0, 1, 0);
        ESP_LOGI(TAG, "sent publish successful, msg_id=%d", msg_id);

        msg_id = esp_mqtt_client_subscribe(client, "/topic/qos0", 0);
        ESP_LOGI(TAG, "sent subscribe successful, msg_id=%d", msg_id);

        msg_id = esp_mqtt_client_subscribe(client, "/topic/qos1", 1);
        ESP_LOGI(TAG, "sent subscribe successful, msg_id=%d", msg_id);

        msg_id = esp_mqtt_client_unsubscribe(client, "/topic/qos1");
        ESP_LOGI(TAG, "sent unsubscribe successful, msg_id=%d", msg_id);
        break;
    case MQTT_EVENT_DISCONNECTED:
        ESP_LOGI(TAG, "MQTT_EVENT_DISCONNECTED");
        break;

    case MQTT_EVENT_SUBSCRIBED:
        ESP_LOGI(TAG, "MQTT_EVENT_SUBSCRIBED, msg_id=%d", event->msg_id);
        msg_id = esp_mqtt_client_publish(client, "/topic/qos0", "data", 0, 0, 0);
        ESP_LOGI(TAG, "sent publish successful, msg_id=%d", msg_id);
    }
```

```
        break;
    case MQTT_EVENT_UNSUBSCRIBED:
        ESP_LOGI(TAG, "MQTT_EVENT_UNSUBSCRIBED, msg_id=%d", event->msg_id);
        break;
    case MQTT_EVENT_PUBLISHED:
        ESP_LOGI(TAG, "MQTT_EVENT_PUBLISHED, msg_id=%d", event->msg_id);
        break;
    case MQTT_EVENT_DATA:
        ESP_LOGI(TAG, "MQTT_EVENT_DATA");
        printf("TOPIC=.*s\r\n", event->topic_len, event->topic);
        printf("DATA=.*s\r\n", event->data_len, event->data);
        break;
    case MQTT_EVENT_ERROR:
        ESP_LOGI(TAG, "MQTT_EVENT_ERROR");
        break;
    default:
        ESP_LOGI(TAG, "Other event id:%d", event->event_id);
        break;
    }
}

static void mqtt_app_start(void)
{
    esp_mqtt_client_config_t mqtt_cfg = {
        .broker.address.uri = CONFIG_BROKER_URL,
    };

    esp_mqtt_client_handle_t client = esp_mqtt_client_init(&mqtt_cfg);
    esp_mqtt_client_register_event(client, ESP_EVENT_ANY_ID, mqtt_event_handler, NULL);
}
```

```
    esp_mqtt_client_start(client);
}

void app_main(void)
{
    ESP_LOGI(TAG, "[APP] Startup..");
    ESP_LOGI(TAG, "[APP] Free memory: %" PRIu32 " bytes", esp_get_free_heap_size());
    ESP_LOGI(TAG, "[APP] IDF version: %s", esp_get_idf_version());

    esp_log_level_set("*", ESP_LOG_INFO);
    esp_log_level_set("mqtt_client", ESP_LOG_VERBOSE);
    esp_log_level_set("MQTT_EXAMPLE", ESP_LOG_VERBOSE);
    esp_log_level_set("TRANSPORT_BASE", ESP_LOG_VERBOSE);
    esp_log_level_set("esp-tls", ESP_LOG_VERBOSE);
    esp_log_level_set("TRANSPORT", ESP_LOG_VERBOSE);
    esp_log_level_set("outbox", ESP_LOG_VERBOSE);

    ESP_ERROR_CHECK(nvs_flash_init());
    ESP_ERROR_CHECK(esp_netif_init());
    ESP_ERROR_CHECK(esp_event_loop_create_default());

    ESP_ERROR_CHECK(example_connect());

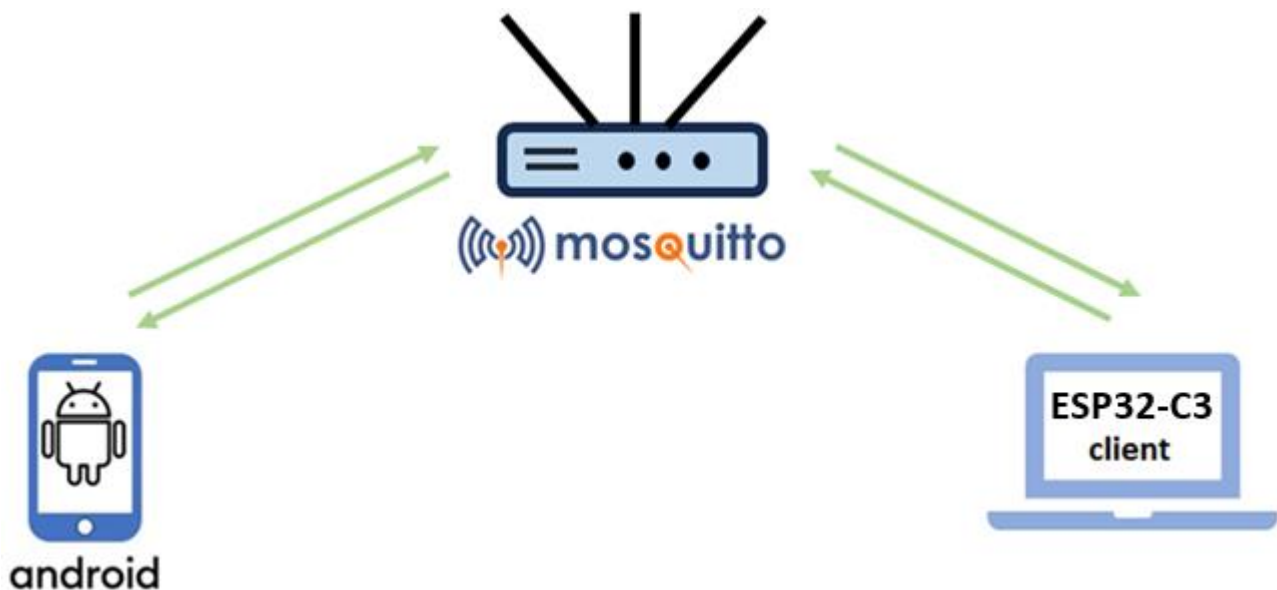
    mqtt_app_start();
}
```

Bước 5: Build project và nạp chương trình vào vi điều khiển



Example 2.11

Mục tiêu: Tạo và quản lý ứng dụng sử dụng MQTT client chạy trên vi điều khiển **ESP32** (không sử dụng **idf.py menuconfig** cấu hình WIFI code) có kiến trúc như sau:



Yêu cầu: Ứng dụng thực hiện các chức năng sau:

- ✓ Sử dụng **MQTT-Explorer** Subscribe với **Broker** với topic có tên **/test/topic**
- ✓ Sử dụng **ESP32** Publish nội dung "**Hello AIoT**" đến **Broker** với topic có tên **/test/topic**
- ✓ Sử dụng **MQTT-Explorer** publish tới **ESP32** nội dung "**Hi from the IoT application**" với topic có tên **/test/topic1**

Hướng dẫn:

Bước 1: Cấu hình **MQTT Broker**

<code>... \mosquitto\mosquitto.conf</code>
<pre>listener 1883 allow_anonymous true</pre>

Bước 2: Sử dụng Visual Studio Code, mở Command Palette (Ctrl+Shift+P) và nhập **IDF: New Project** để bắt đầu tạo project với các thông tin như sau:

- ✓ Project Name: **example211**
- ✓ Enter Project directory: **D:\java-projects**
- ✓ Choose ESP-IDF Board: **ESP32-C3 chip(via ESP USB Bridge)**
- ✓ Choose serial port: **COMx**

Bước 3: Sử dụng **MQTT-Explorer** đăng ký topic có tên **/test/topic** với Broker

+

Connections

example101

mqtt://localhost:1883/

test.mosquitto.org

mqtt://test.mosquitto.org:1883/

MQTT Connection

mqtt://localhost:1883/

Topic

/test/topic

QoS

0

+ ADD

Topic	QoS
-------	-----

MQTT Client ID

mqtt-explorer-63d325c9

CERTIFICATES

BACK

Bước 4: Sửa file **app_main.c** như sau:**app_main.c**

```
#include <stdio.h>
#include <stdint.h>
#include <stddef.h>
#include <string.h>
#include "esp_wifi.h"
#include "esp_system.h"
#include "nvs_flash.h"
#include "esp_event.h"
#include "esp_netif.h"
#include "protocol_examples_common.h"

#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/semphr.h"
#include "freertos/queue.h"

#include "lwip/sockets.h"
#include "lwip/dns.h"
#include "lwip/netdb.h"

#include "esp_log.h"
#include "mqtt_client.h"

static const char *TAG = "MQTT_EXAMPLE";
#define ESP_WIFI_SSID "YOUR_SSID"
#define ESP_WIFI_PASS "YOUR_PASSWORD"
#define ESP_BROKER_IP "BROKER_IP" //mqtt://192.168.1.4:1883

uint32_t MQTT_CONNECTED = 0;
static void mqtt_app_start(void);
```



```
static void wifi_event_handler(void *arg, esp_event_base_t event_base, int32_t event_id, void *event_data)
{
    if (event_base == WIFI_EVENT)
    {
        switch (event_id)
        {
            case WIFI_EVENT_STA_START:
                esp_wifi_connect();
                ESP_LOGI(TAG, "Trying to connect with Wi-Fi");
                break;
            case WIFI_EVENT_STA_DISCONNECTED:
                ESP_LOGI(TAG, "Disconnected: Retrying Wi-Fi");
                esp_wifi_connect();
                break;
            default:
                break;
        }
    }
    else if (event_base == IP_EVENT && event_id == IP_EVENT_STA_GOT_IP)
    {
        ESP_LOGI(TAG, "Got IP: Starting MQTT Client");
        mqtt_app_start();
    }
}

void wifi_init(void)
{
    ESP_ERROR_CHECK(esp_netif_init());
    ESP_ERROR_CHECK(esp_event_loop_create_default());
    esp_netif_create_default_wifi_sta();

    wifi_init_config_t cfg = WIFI_INIT_CONFIG_DEFAULT();
    ESP_ERROR_CHECK(esp_wifi_init(&cfg));
}
```

```
ESP_ERROR_CHECK(esp_event_handler_instance_register(WIFI_EVENT, ESP_EVENT_ANY_ID, &wifi_event_handler, NULL, NULL));
ESP_ERROR_CHECK(esp_event_handler_instance_register(IP_EVENT, IP_EVENT_STA_GOT_IP, &wifi_event_handler, NULL, NULL));

wifi_config_t wifi_config = {
    .sta = {
        .ssid = ESP_WIFI_SSID,
        .password = ESP_WIFI_PASS,
        .threshold.authmode = WIFI_AUTH_WPA2_PSK,
    },
};
ESP_ERROR_CHECK(esp_wifi_set_mode(WIFI_MODE_STA));
ESP_ERROR_CHECK(esp_wifi_set_config(WIFI_IF_STA, &wifi_config));
ESP_ERROR_CHECK(esp_wifi_start());
}

static void mqtt_event_handler(void *handler_args, esp_event_base_t base, int32_t event_id, void *event_data)
{
    ESP_LOGD(TAG, "Event dispatched from event loop base=%s, event_id=%ld", base, (long)event_id);
    esp_mqtt_event_handle_t event = event_data;
    esp_mqtt_client_handle_t client = event->client;
    int msg_id;
    switch ((esp_mqtt_event_id_t)event_id)
    {
        case MQTT_EVENT_CONNECTED:
            ESP_LOGI(TAG, "MQTT_EVENT_CONNECTED");
            MQTT_CONNECTED = 1;
            msg_id = esp_mqtt_client_subscribe(client, "/test/topic1", 0);
            ESP_LOGI(TAG, "Subscribed, msg_id=%d", msg_id);
            break;
        case MQTT_EVENT_DISCONNECTED:
            ESP_LOGI(TAG, "MQTT_EVENT_DISCONNECTED");
            MQTT_CONNECTED = 0;
    }
}
```

```
        break;
    case MQTT_EVENT_DATA:
        ESP_LOGI(TAG, "MQTT_EVENT_DATA");
        printf("TOPIC=.%s\r\n", event->topic_len, event->topic);
        printf("DATA=.%s\r\n", event->data_len, event->data);
        break;
    default:
        ESP_LOGI(TAG, "Other event id:%ld", (long)event->event_id);
        break;
    }
}

esp_mqtt_client_handle_t client = NULL;
static void mqtt_app_start(void)
{
    ESP_LOGI(TAG, "STARTING MQTT");
    esp_mqtt_client_config_t mqttConfig = {
        .broker.address.uri = ESP_BROKER_IP,
    };
    client = esp_mqtt_client_init(&mqttConfig);
    esp_mqtt_client_register_event(client, ESP_EVENT_ANY_ID, mqtt_event_handler, client);
    esp_mqtt_client_start(client);
}

void Publisher_Task(void *params)
{
    while (true)
    {
        if (MQTT_CONNECTED)
        {
            esp_mqtt_client_publish(client, "/test/topic", "Hello AIoT", 0, 0, 0);
            vTaskDelay(1500 / portTICK_PERIOD_MS);
        }
    }
}
```

```
    }  
}  
  
void app_main(void)  
{  
    esp_err_t ret = nvs_flash_init();  
    if (ret == ESP_ERR_NVS_NO_FREE_PAGES || ret == ESP_ERR_NVS_NEW_VERSION_FOUND)  
    {  
        ESP_ERROR_CHECK(nvs_flash_erase());  
        ret = nvs_flash_init();  
    }  
    ESP_ERROR_CHECK(ret);  
    wifi_init();  
    xTaskCreate(Publisher_Task, "Publisher_Task", 1024 * 5, NULL, 5, NULL);  
}
```

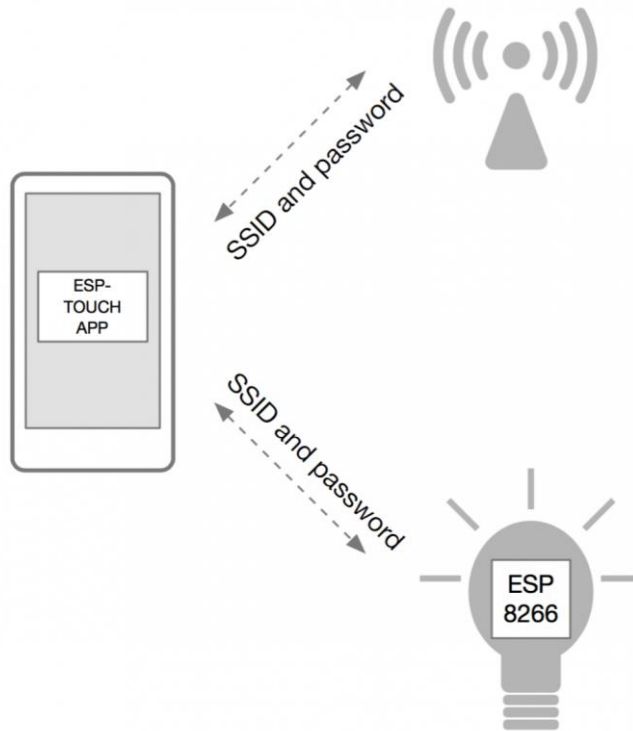
Bước 5: Build project và nạp chương trình vào vi điều khiển



Bước 6: Sử dụng **MQTT-Explorer** publish tới Broker nội dung **"Hi from the IoT application"** với topic có tên **/test/topic1**

Example 2.12

Mục tiêu: Tạo và quản lý ứng dụng sử dụng MQTT client chạy trên vi điều khiển ESP32 (cấu hình WIFI thông qua App, gửi thông tin SSID & Password qua sóng WiFi dưới dạng broadcast hoặc multicast) có kiến trúc như sau:



Yêu cầu: Ứng dụng thực hiện các chức năng sau:

- ✓ ESP32 khởi động và vào chế độ SmartConfig.
- ✓ Ứng dụng trên điện thoại ESP-Touch gửi thông tin SSID & Password qua sóng WiFi dưới dạng broadcast hoặc multicast.
- ✓ ESP32 bắt tín hiệu và giải mã thông tin này để kết nối với WiFi.
- ✓ Khi kết nối thành công, ESP32 có thể lưu lại thông tin WiFi vào flash để sử dụng lần sau.

Hướng dẫn:

Bước 1: Sử dụng Visual Studio Code, mở Command Palette (Ctrl+Shift+P) và nhập **IDF: New Project**) để bắt đầu tạo project với các thông tin như sau:

- ✓ Project Name: **example212**
- ✓ Enter Project directory: **D:\java-projects**
- ✓ Choose ESP-IDF Board: **ESP32-C3 chip(via ESP USB Bridge)**
- ✓ Choose serial port: COMx

Bước 2: Build app **EspTouch** từ source code:

- ✓ Android: <https://github.com/EsspressifApp/EsptouchForAndroid>
- ✓ iOS: <https://github.com/EsspressifApp/EsptouchForIOS>

Bước 3: Sửa file **app_main.c** như sau:**app_main.c**

```
#include <string.h>
#include <stdlib.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"
#include "freertos/event_groups.h"
#include "esp_wifi.h"
#include "esp_eap_client.h"
#include "esp_event.h"
#include "esp_log.h"
#include "esp_system.h"
#include "nvs_flash.h"
#include "esp_netif.h"
#include "esp_smartconfig.h"
#include "esp_mac.h"

static EventGroupHandle_t s_wifi_event_group;

static const int CONNECTED_BIT = BIT0;
static const int ESPTOUCH_DONE_BIT = BIT1;
static const char *TAG = "smartconfig_example";

static void smartconfig_example_task(void *parm);

static void event_handler(void *arg, esp_event_base_t event_base,
                          int32_t event_id, void *event_data)
{
    if (event_base == WIFI_EVENT && event_id == WIFI_EVENT_STA_START)
    {
        xTaskCreate(smartconfig_example_task, "smartconfig_example_task", 4096, NULL, 3, NULL);
    }
    else if (event_base == WIFI_EVENT && event_id == WIFI_EVENT_STA_DISCONNECTED)
```

```
{
    esp_wifi_connect();
    xEventGroupClearBits(s_wifi_event_group, CONNECTED_BIT);
}
else if (event_base == IP_EVENT && event_id == IP_EVENT_STA_GOT_IP)
{
    xEventGroupSetBits(s_wifi_event_group, CONNECTED_BIT);
}
else if (event_base == SC_EVENT && event_id == SC_EVENT_SCAN_DONE)
{
    ESP_LOGI(TAG, "Scan done");
}
else if (event_base == SC_EVENT && event_id == SC_EVENT_FOUND_CHANNEL)
{
    ESP_LOGI(TAG, "Found channel");
}
else if (event_base == SC_EVENT && event_id == SC_EVENT_GOT_SSID_PSWD)
{
    ESP_LOGI(TAG, "Got SSID and password");

    smartconfig_event_got_ssid_pswd_t *evt = (smartconfig_event_got_ssid_pswd_t *)event_data;
    wifi_config_t wifi_config;
    uint8_t ssid[33] = {0};
    uint8_t password[65] = {0};
    uint8_t rvd_data[33] = {0};

    bzero(&wifi_config, sizeof(wifi_config_t));
    memcpy(wifi_config.sta.ssid, evt->ssid, sizeof(wifi_config.sta.ssid));
    memcpy(wifi_config.sta.password, evt->password, sizeof(wifi_config.sta.password));

#ifdef CONFIG_SET_MAC_ADDRESS_OF_TARGET_AP
    wifi_config.sta.bssid_set = evt->bssid_set;
    if (wifi_config.sta.bssid_set == true)
```

```
{
    ESP_LOGI(TAG, "Set MAC address of target AP: " MACSTR " ", MAC2STR(evt->bssid));
    memcpy(wifi_config.sta.bssid, evt->bssid, sizeof(wifi_config.sta.bssid));
}
#endif

memcpy(ssid, evt->ssid, sizeof(evt->ssid));
memcpy(password, evt->password, sizeof(evt->password));
ESP_LOGI(TAG, "SSID:%s", ssid);
ESP_LOGI(TAG, "PASSWORD:%s", password);
if (evt->type == SC_TYPE_ESPTOUCH_V2)
{
    ESP_ERROR_CHECK(esp_smartconfig_get_rvd_data(rvd_data, sizeof(rvd_data)));
    ESP_LOGI(TAG, "RVD_DATA:");
    for (int i = 0; i < 33; i++)
    {
        printf("%02x ", rvd_data[i]);
    }
    printf("\n");
}

ESP_ERROR_CHECK(esp_wifi_disconnect());
ESP_ERROR_CHECK(esp_wifi_set_config(WIFI_IF_STA, &wifi_config));
esp_wifi_connect();
}
else if (event_base == SC_EVENT && event_id == SC_EVENT_SEND_ACK_DONE)
{
    xEventGroupSetBits(s_wifi_event_group, ESPTOUCH_DONE_BIT);
}
}

static void initialise_wifi(void)
{
```



```
ESP_ERROR_CHECK(esp_netif_init());
s_wifi_event_group = xEventGroupCreate();
ESP_ERROR_CHECK(esp_event_loop_create_default());
esp_netif_t *sta_netif = esp_netif_create_default_wifi_sta();
assert(sta_netif);

wifi_init_config_t cfg = WIFI_INIT_CONFIG_DEFAULT();
ESP_ERROR_CHECK(esp_wifi_init(&cfg));

ESP_ERROR_CHECK(esp_event_handler_register(WIFI_EVENT, ESP_EVENT_ANY_ID, &event_handler, NULL));
ESP_ERROR_CHECK(esp_event_handler_register(IP_EVENT, IP_EVENT_STA_GOT_IP, &event_handler, NULL));
ESP_ERROR_CHECK(esp_event_handler_register(SC_EVENT, ESP_EVENT_ANY_ID, &event_handler, NULL));

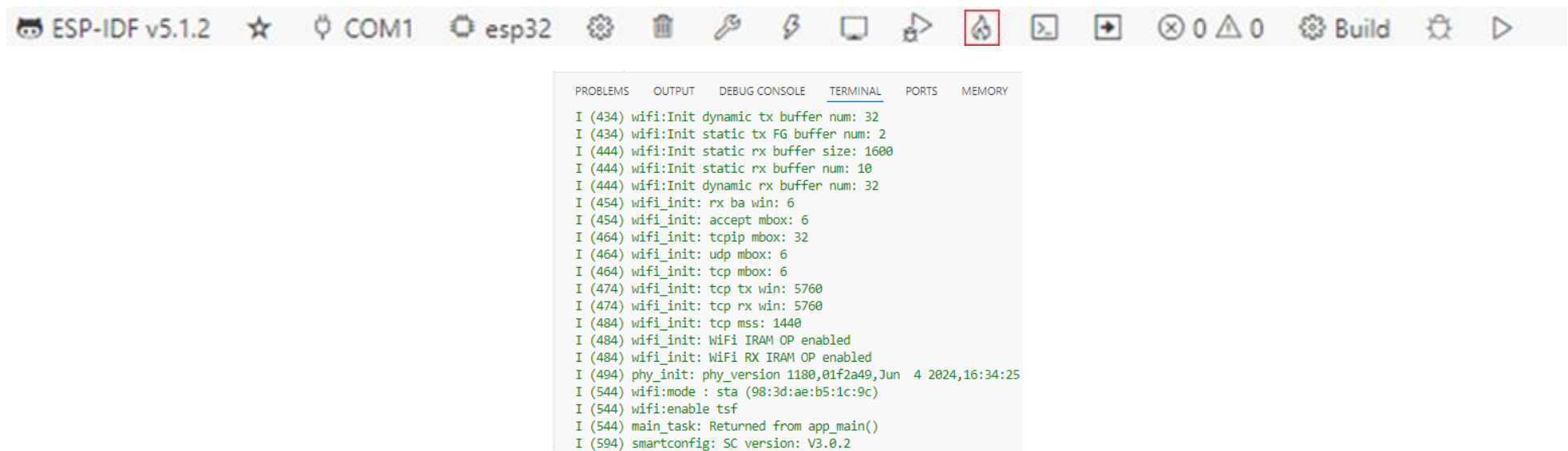
ESP_ERROR_CHECK(esp_wifi_set_mode(WIFI_MODE_STA));
ESP_ERROR_CHECK(esp_wifi_start());
}

static void smartconfig_example_task(void *parm)
{
    EventBits_t uxBits;
    ESP_ERROR_CHECK(esp_smartconfig_set_type(SC_TYPE_ESPTOUCH));
    smartconfig_start_config_t cfg = SMARTCONFIG_START_CONFIG_DEFAULT();
    ESP_ERROR_CHECK(esp_smartconfig_start(&cfg));
    while (1)
    {
        uxBits = xEventGroupWaitBits(s_wifi_event_group, CONNECTED_BIT | ESPTOUCH_DONE_BIT, true, false, portMAX_DELAY);
        if (uxBits & CONNECTED_BIT)
        {
            ESP_LOGI(TAG, "WiFi Connected to ap");
        }
        if (uxBits & ESPTOUCH_DONE_BIT)
        {
            ESP_LOGI(TAG, "smartconfig over");
        }
    }
}
```

```
        esp_smartconfig_stop();
        vTaskDelete(NULL);
    }
}

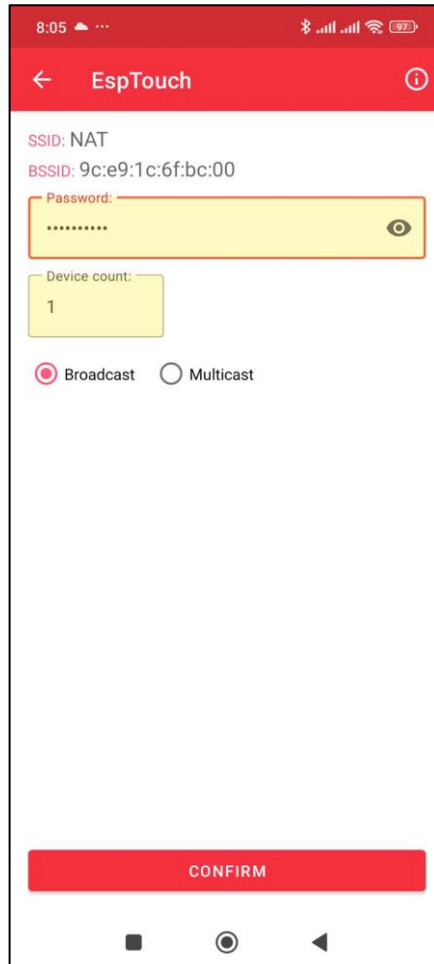
void app_main(void)
{
    ESP_ERROR_CHECK(nvs_flash_init());
    initialise_wifi();
}
```

Bước 4: Build project và nạp chương trình vào vi điều khiển, sau đó theo dõi tại màn hình Terminal chúng ta sẽ thấy như sau:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  MEMORY
I (434) wifi:Init dynamic tx buffer num: 32
I (434) wifi:Init static tx FG buffer num: 2
I (444) wifi:Init static rx buffer size: 1600
I (444) wifi:Init static rx buffer num: 10
I (444) wifi:Init dynamic rx buffer num: 32
I (454) wifi_init: rx ba win: 6
I (454) wifi_init: accept mbox: 6
I (464) wifi_init: tcpip mbox: 32
I (464) wifi_init: udp mbox: 6
I (464) wifi_init: tcp mbox: 6
I (474) wifi_init: tcp tx win: 5760
I (474) wifi_init: tcp rx win: 5760
I (484) wifi_init: tcp mss: 1440
I (484) wifi_init: WiFi IRAM OP enabled
I (484) wifi_init: WiFi RX IRAM OP enabled
I (494) phy_init: phy_version 1180,01f2a49,Jun  4 2024,16:34:25
I (544) wifi:mode : sta (98:3d:ae:b5:1c:9c)
I (544) wifi:enable tsf
I (544) main_task: Returned from app_main()
I (594) smartconfig: SC version: V3.0.2
```

Bước 5: Sử dụng App **EspTouch** trên Android nhập **Password** của WIFI → **CONFIRM** → sau đó theo dõi tại màn hình Terminal ESP32 sẽ kết nối WiFi với **SSID** và **Password** được gửi từ app



```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  MEMORY  XRTOS  ESP-IDF

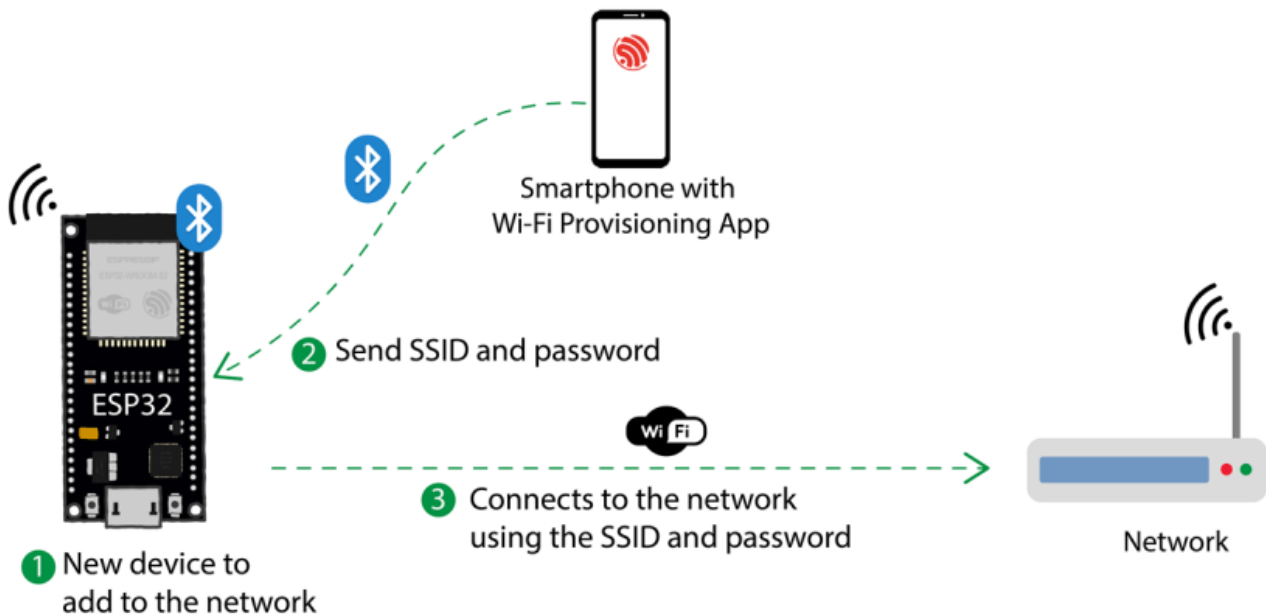
I (5414) smartconfig: Start to find channel...
I (5414) smartconfig_example: Scan done
I (725404) smartconfig: TYPE: ESPTOUCH
I (725404) smartconfig: T|PHONE MAC:b2:88:67:9b:d2:cf
I (725414) smartconfig: T|AP MAC:9c:e9:1c:6f:bc:00
I (725414) smartconfig: Found channel on 4-0. Start to get ssid and password...
I (725414) smartconfig_example: Found channel
I (733694) smartconfig: T|pswd: 0902880088
I (733694) smartconfig: T|ssid: NAT
I (733694) smartconfig: T|bssid: 9c:e9:1c:6f:bc:00
I (733694) wifi:ic_disable_sniffer
I (733704) smartconfig_example: Got SSID and password
I (733704) smartconfig_example: Set MAC address of target AP: 9c:e9:1c:6f:bc:00
I (733714) smartconfig_example: SSID:NAT
I (733724) smartconfig_example: PASSWORD:0902880088
W (733724) wifi:Password length matches WPA2 standards, authmode threshold changes from OPEN to WPA2
I (733744) wifi:new:<4,0>, old:<4,0>, ap:<255,255>, sta:<4,0>, prof:1, snd_ch_cfg:0x0
I (733744) wifi:state: init -> auth (0xb0)
I (733754) wifi:state: auth -> assoc (0x0)
I (733764) wifi:state: assoc -> run (0x10)
I (735864) wifi:connected with NAT, aid = 9, channel 4, BW20, bssid = 9c:e9:1c:6f:bc:00
I (735864) wifi:security: WPA2-PSK, phy: bgn, rssi: -65
I (735864) wifi:pm start, type: 1

I (735864) wifi:dp: 1, bi: 102400, li: 3, scale listen interval from 307200 us to 307200 us
I (735874) wifi:set rx beacon pti, rx_bcn_pti: 0, bcn_timeout: 25000, mt_pti: 0, mt_time: 10000
I (735894) wifi:AP's beacon interval = 102400 us, DTIM period = 1
I (737914) wifi:<ba-add>idx:0 (ifx:0, 9c:e9:1c:6f:bc:00), tid:0, ssn:0, winSize:64
I (738024) wifi:<ba-add>idx:1 (ifx:0, 9c:e9:1c:6f:bc:00), tid:6, ssn:4, winSize:64
I (738884) esp_netif_handlers: sta ip: 192.168.1.8, mask: 255.255.255.0, gw: 192.168.1.1

```

Example 2.13

Mục tiêu: Tạo và quản lý ứng dụng sử dụng MQTT client chạy trên vi điều khiển **ESP32** (cấu hình WIFI qua App thiết bị di động sử dụng Bluetooth) có kiến trúc như sau:



Yêu cầu: Ứng dụng thực hiện các chức năng sau:

- ✓ ESP32 in ra QR code
- ✓ App quét QR Code → nhập mã PIN để cấu hình thông số WIFI

Hướng dẫn:

Bước 1: Sử dụng Visual Studio Code, mở Command Palette (Ctrl+Shift+P) và nhập **IDF: New Project**) để bắt đầu tạo project với các thông tin như sau:

- ✓ Project Name: **example213**
- ✓ Enter Project directory: **D:\java-projects**
- ✓ Choose ESP-IDF Board: **ESP32-C3 chip(via ESP USB Bridge)**
- ✓ Choose serial port: COMx

Bước 2: Build app **ESP BLE Provisioning** từ source code:

- ✓ Android: <https://github.com/espressif/esp-idf-provisioning-android.git>
- ✓ iOS: <https://github.com/espressif/esp-idf-provisioning-ios.git>

Bước 3: Sửa file **app_main.c** như sau:**app_main.c**

```
#include <stdio.h>
#include <string.h>

#include <freertos/FreeRTOS.h>
#include <freertos/task.h>
#include <freertos/event_groups.h>

#include <esp_log.h>
#include <esp_wifi.h>
#include <esp_event.h>
#include <nvs_flash.h>

#include <wifi_provisioning/manager.h>

#include <wifi_provisioning/scheme_ble.h>

#include "qrcode.h"

static const char *TAG = "app";

#define EXAMPLE_PROV_SEC2_USERNAME "wifiprov"
#define EXAMPLE_PROV_SEC2_PWD "abcd1234"

static const char sec2_salt[] = {
    0x03, 0x6e, 0xe0, 0xc7, 0xbc, 0xb9, 0xed, 0xa8, 0x4c, 0x9e, 0xac, 0x97, 0xd9, 0x3d, 0xec, 0xf4};

static const char sec2_verifier[] = {
    0x7c, 0x7c, 0x85, 0x47, 0x65, 0x08, 0x94, 0x6d, 0xd6, 0x36, 0xaf, 0x37, 0xd7, 0xe8, 0x91, 0x43,
```

```

0x78, 0xcf, 0xfd, 0x61, 0x6c, 0x59, 0xd2, 0xf8, 0x39, 0x08, 0x12, 0x72, 0x38, 0xde, 0x9e, 0x24,
0xa4, 0x70, 0x26, 0x1c, 0xdf, 0xa9, 0x03, 0xc2, 0xb2, 0x70, 0xe7, 0xb1, 0x32, 0x24, 0xda, 0x11,
0x1d, 0x97, 0x18, 0xdc, 0x60, 0x72, 0x08, 0xcc, 0x9a, 0xc9, 0x0c, 0x48, 0x27, 0xe2, 0xae, 0x89,
0xaa, 0x16, 0x25, 0xb8, 0x04, 0xd2, 0x1a, 0x9b, 0x3a, 0x8f, 0x37, 0xf6, 0xe4, 0x3a, 0x71, 0x2e,
0xe1, 0x27, 0x86, 0x6e, 0xad, 0xce, 0x28, 0xff, 0x54, 0x46, 0x60, 0x1f, 0xb9, 0x96, 0x87, 0xdc,
0x57, 0x40, 0xa7, 0xd4, 0x6c, 0xc9, 0x77, 0x54, 0xdc, 0x16, 0x82, 0xf0, 0xed, 0x35, 0x6a, 0xc4,
0x70, 0xad, 0x3d, 0x90, 0xb5, 0x81, 0x94, 0x70, 0xd7, 0xbc, 0x65, 0xb2, 0xd5, 0x18, 0xe0, 0x2e,
0xc3, 0xa5, 0xf9, 0x68, 0xdd, 0x64, 0x7b, 0xb8, 0xb7, 0x3c, 0x9c, 0xfc, 0x00, 0xd8, 0x71, 0x7e,
0xb7, 0x9a, 0x7c, 0xb1, 0xb7, 0xc2, 0xc3, 0x18, 0x34, 0x29, 0x32, 0x43, 0x3e, 0x00, 0x99, 0xe9,
0x82, 0x94, 0xe3, 0xd8, 0x2a, 0xb0, 0x96, 0x29, 0xb7, 0xdf, 0x0e, 0x5f, 0x08, 0x33, 0x40, 0x76,
0x52, 0x91, 0x32, 0x00, 0x9f, 0x97, 0x2c, 0x89, 0x6c, 0x39, 0x1e, 0xc8, 0x28, 0x05, 0x44, 0x17,
0x3f, 0x68, 0x02, 0x8a, 0x9f, 0x44, 0x61, 0xd1, 0xf5, 0xa1, 0x7e, 0x5a, 0x70, 0xd2, 0xc7, 0x23,
0x81, 0xcb, 0x38, 0x68, 0xe4, 0x2c, 0x20, 0xbc, 0x40, 0x57, 0x76, 0x17, 0xbd, 0x08, 0xb8, 0x96,
0xbc, 0x26, 0xeb, 0x32, 0x46, 0x69, 0x35, 0x05, 0x8c, 0x15, 0x70, 0xd9, 0x1b, 0xe9, 0xbe, 0xcc,
0xa9, 0x38, 0xa6, 0x67, 0xf0, 0xad, 0x50, 0x13, 0x19, 0x72, 0x64, 0xbf, 0x52, 0xc2, 0x34, 0xe2,
0x1b, 0x11, 0x79, 0x74, 0x72, 0xbd, 0x34, 0x5b, 0xb1, 0xe2, 0xfd, 0x66, 0x73, 0xfe, 0x71, 0x64,
0x74, 0xd0, 0x4e, 0xbc, 0x51, 0x24, 0x19, 0x40, 0x87, 0x0e, 0x92, 0x40, 0xe6, 0x21, 0xe7, 0x2d,
0x4e, 0x37, 0x76, 0x2f, 0x2e, 0xe2, 0x68, 0xc7, 0x89, 0xe8, 0x32, 0x13, 0x42, 0x06, 0x84, 0x84,
0x53, 0x4a, 0xb3, 0x0c, 0x1b, 0x4c, 0x8d, 0x1c, 0x51, 0x97, 0x19, 0xab, 0xae, 0x77, 0xff, 0xdb,
0xec, 0xf0, 0x10, 0x95, 0x34, 0x33, 0x6b, 0xcb, 0x3e, 0x84, 0x0f, 0xb9, 0xd8, 0x5f, 0xb8, 0xa0,
0xb8, 0x55, 0x53, 0x3e, 0x70, 0xf7, 0x18, 0xf5, 0xce, 0x7b, 0x4e, 0xbf, 0x27, 0xce, 0xce, 0xa8,
0xb3, 0xbe, 0x40, 0xc5, 0xc5, 0x32, 0x29, 0x3e, 0x71, 0x64, 0x9e, 0xde, 0x8c, 0xf6, 0x75, 0xa1,
0xe6, 0xf6, 0x53, 0xc8, 0x31, 0xa8, 0x78, 0xde, 0x50, 0x40, 0xf7, 0x62, 0xde, 0x36, 0xb2, 0xba};

```

```

static esp_err_t example_get_sec2_salt(const char **salt, uint16_t *salt_len)
{
    ESP_LOGI(TAG, "Development mode: using hard coded salt");
    *salt = sec2_salt;
    *salt_len = sizeof(sec2_salt);
    return ESP_OK;
}

```

```
}

static esp_err_t example_get_sec2_verifier(const char **verifier, uint16_t *verifier_len)
{
    ESP_LOGI(TAG, "Development mode: using hard coded verifier");
    *verifier = sec2_verifier;
    *verifier_len = sizeof(sec2_verifier);
    return ESP_OK;
}

const int WIFI_CONNECTED_EVENT = BIT0;
static EventGroupHandle_t wifi_event_group;

#define PROV_QR_VERSION "v1"
#define PROV_TRANSPORT_SOFTAP "softap"
#define PROV_TRANSPORT_BLE "ble"
#define QRCODE_BASE_URL "https://espressif.github.io/esp-jumpstart/qrcode.html"

static void event_handler(void *arg, esp_event_base_t event_base,
                          int32_t event_id, void *event_data)
{
    static int retries;

    if (event_base == WIFI_PROV_EVENT)
    {
        switch (event_id)
        {
            case WIFI_PROV_START:
                ESP_LOGI(TAG, "Provisioning started");
                break;
        }
    }
}
```

```
case WIFI_PROV_CRED_RECV:
{
    wifi_sta_config_t *wifi_sta_cfg = (wifi_sta_config_t *)event_data;
    ESP_LOGI(TAG, "Received Wi-Fi credentials"
                "\n\tSSID      : %s\n\tPassword : %s",
                (const char *)wifi_sta_cfg->ssid,
                (const char *)wifi_sta_cfg->password);
    break;
}
case WIFI_PROV_CRED_FAIL:
{
    wifi_prov_sta_fail_reason_t *reason = (wifi_prov_sta_fail_reason_t *)event_data;
    ESP_LOGE(TAG, "Provisioning failed!\n\tReason : %s"
                "\n\tPlease reset to factory and retry provisioning",
                (*reason == WIFI_PROV_STA_AUTH_ERROR) ? "Wi-Fi station authentication failed" : "Wi-Fi access-point not
found");

    retries++;
    if (retries >= CONFIG_EXAMPLE_PROV_MGR_MAX_RETRY_CNT)
    {
        ESP_LOGI(TAG, "Failed to connect with provisioned AP, resetting provisioned credentials");
        wifi_prov_mgr_reset_sm_state_on_failure();
        retries = 0;
    }

    break;
}
case WIFI_PROV_CRED_SUCCESS:
    ESP_LOGI(TAG, "Provisioning successful");

    retries = 0;
```



```
        break;
    case WIFI_PROV_END:
        wifi_prov_mgr_deinit();
        break;
    default:
        break;
    }
}
else if (event_base == WIFI_EVENT)
{
    switch (event_id)
    {
    case WIFI_EVENT_STA_START:
        esp_wifi_connect();
        break;
    case WIFI_EVENT_STA_DISCONNECTED:
        ESP_LOGI(TAG, "Disconnected. Connecting to the AP again...");
        esp_wifi_connect();
        break;
    default:
        break;
    }
}
else if (event_base == IP_EVENT && event_id == IP_EVENT_STA_GOT_IP)
{
    ip_event_got_ip_t *event = (ip_event_got_ip_t *)event_data;
    ESP_LOGI(TAG, "Connected with IP Address:" IPSTR, IP2STR(&event->ip_info.ip));
    xEventGroupSetBits(wifi_event_group, WIFI_CONNECTED_EVENT);
}
else if (event_base == PROTOCOMM_TRANSPORT_BLE_EVENT)
```

```
{
    switch (event_id)
    {
        case PROTOCOMM_TRANSPORT_BLE_CONNECTED:
            ESP_LOGI(TAG, "BLE transport: Connected!");
            break;
        case PROTOCOMM_TRANSPORT_BLE_DISCONNECTED:
            ESP_LOGI(TAG, "BLE transport: Disconnected!");
            break;
        default:
            break;
    }
}
else if (event_base == PROTOCOMM_SECURITY_SESSION_EVENT)
{
    switch (event_id)
    {
        case PROTOCOMM_SECURITY_SESSION_SETUP_OK:
            ESP_LOGI(TAG, "Secured session established!");
            break;
        case PROTOCOMM_SECURITY_SESSION_INVALID_SECURITY_PARAMS:
            ESP_LOGE(TAG, "Received invalid security parameters for establishing secure session!");
            break;
        case PROTOCOMM_SECURITY_SESSION_CREDENTIALS_MISMATCH:
            ESP_LOGE(TAG, "Received incorrect username and/or PoP for establishing secure session!");
            break;
        default:
            break;
    }
}
}
```

```
static void wifi_init_sta(void)
{
    ESP_ERROR_CHECK(esp_wifi_set_mode(WIFI_MODE_STA));
    ESP_ERROR_CHECK(esp_wifi_start());
}

static void get_device_service_name(char *service_name, size_t max)
{
    uint8_t eth_mac[6];
    const char *ssid_prefix = "PROV_";
    esp_wifi_get_mac(WIFI_IF_STA, eth_mac);
    snprintf(service_name, max, "%s%02X%02X%02X",
             ssid_prefix, eth_mac[3], eth_mac[4], eth_mac[5]);
}

esp_err_t custom_prov_data_handler(uint32_t session_id, const uint8_t *inbuf, ssize_t inlen,
                                   uint8_t **outbuf, ssize_t *outlen, void *priv_data)
{
    if (inbuf)
    {
        ESP_LOGI(TAG, "Received data: %.*s", inlen, (char *)inbuf);
    }
    char response[] = "SUCCESS";
    *outbuf = (uint8_t *)strdup(response);
    if (*outbuf == NULL)
    {
        ESP_LOGE(TAG, "System out of memory");
        return ESP_ERR_NO_MEM;
    }
    *outlen = strlen(response) + 1;
}
```

```
    return ESP_OK;
}

static void wifi_prov_print_qr(const char *name, const char *username, const char *pop, const char *transport)
{
    if (!name || !transport)
    {
        ESP_LOGW(TAG, "Cannot generate QR code payload. Data missing.");
        return;
    }
    char payload[150] = {0};
    if (pop)
    {
        snprintf(payload, sizeof(payload), "{\"ver\":\"%s\", \"name\":\"%s\", \"username\":\"%s\", \"pop\":\"%s\", \"transport\":\"%s\"}",
            PROV_QR_VERSION, name, username, pop, transport);
    }
    else
    {
        snprintf(payload, sizeof(payload), "{\"ver\":\"%s\", \"name\":\"%s\", \"transport\":\"%s\"}",
            PROV_QR_VERSION, name, transport);
    }

    ESP_LOGI(TAG, "Scan this QR code from the provisioning application for Provisioning.");
    esp_qrcode_config_t cfg = ESP_QRCODE_CONFIG_DEFAULT();
    esp_qrcode_generate(&cfg, payload);
    ESP_LOGI(TAG, "If QR code is not visible, copy paste the below URL in a browser.\n%s?data=%s", QRCODE_BASE_URL,
payload);
}
```

```
void app_main(void)
{
    esp_err_t ret = nvs_flash_init();
    if (ret == ESP_ERR_NVS_NO_FREE_PAGES || ret == ESP_ERR_NVS_NEW_VERSION_FOUND)
    {

        ESP_ERROR_CHECK(nvs_flash_erase());

        ESP_ERROR_CHECK(nvs_flash_init());
    }

    ESP_ERROR_CHECK(esp_netif_init());

    ESP_ERROR_CHECK(esp_event_loop_create_default());
    wifi_event_group = xEventGroupCreate();

    ESP_ERROR_CHECK(esp_event_handler_register(WIFI_PROV_EVENT, ESP_EVENT_ANY_ID, &event_handler, NULL));

    ESP_ERROR_CHECK(esp_event_handler_register(PROTOCOMM_TRANSPORT_BLE_EVENT, ESP_EVENT_ANY_ID, &event_handler, NULL));

    ESP_ERROR_CHECK(esp_event_handler_register(PROTOCOMM_SECURITY_SESSION_EVENT, ESP_EVENT_ANY_ID, &event_handler, NULL));
    ESP_ERROR_CHECK(esp_event_handler_register(WIFI_EVENT, ESP_EVENT_ANY_ID, &event_handler, NULL));
    ESP_ERROR_CHECK(esp_event_handler_register(IP_EVENT, IP_EVENT_STA_GOT_IP, &event_handler, NULL));

    esp_netif_create_default_wifi_sta();
    wifi_init_config_t cfg = WIFI_INIT_CONFIG_DEFAULT();
    ESP_ERROR_CHECK(esp_wifi_init(&cfg));

    wifi_prov_mgr_config_t config = {

        .scheme = wifi_prov_scheme_ble,
```

```
.scheme_event_handler = WIFI_PROV_SCHEME_BLE_EVENT_HANDLER_FREE_BTDM

};
ESP_ERROR_CHECK(wifi_prov_mgr_init(config));

bool provisioned = false;
wifi_prov_mgr_reset_provisioning();

if (!provisioned)
{
    ESP_LOGI(TAG, "Starting provisioning");
    char service_name[12];
    get_device_service_name(service_name, sizeof(service_name));

    wifi_prov_security_t security = WIFI_PROV_SECURITY_2;

    const char *username = EXAMPLE_PROV_SEC2_USERNAME;
    const char *pop = EXAMPLE_PROV_SEC2_PWD;

    wifi_prov_security2_params_t sec2_params = {};

    ESP_ERROR_CHECK(example_get_sec2_salt(&sec2_params.salt, &sec2_params.salt_len));
    ESP_ERROR_CHECK(example_get_sec2_verifier(&sec2_params.verifier, &sec2_params.verifier_len));

    wifi_prov_security2_params_t *sec_params = &sec2_params;

    const char *service_key = NULL;

    uint8_t custom_service_uuid[] = {
        0xb4,
```

```
        0xdf,  
        0x5a,  
        0x1c,  
        0x3f,  
        0x6b,  
        0xf4,  
        0xbf,  
        0xea,  
        0x4a,  
        0x82,  
        0x03,  
        0x04,  
        0x90,  
        0x1a,  
        0x02,  
};  
  
wifi_prov_scheme_ble_set_service_uuid(custom_service_uuid);  
  
wifi_prov_mgr_endpoint_create("custom-data");  
  
wifi_prov_mgr_disable_auto_stop(1000);  
  
ESP_ERROR_CHECK(wifi_prov_mgr_start_provisioning(security, (const void *)sec_params, service_name, service_key));  
wifi_prov_mgr_endpoint_register("custom-data", custom_prov_data_handler, NULL);  
  
wifi_prov_print_qr(service_name, username, pop, PROV_TRANSPORT_BLE);  
}  
else  
{  
    ESP_LOGI(TAG, "Already provisioned, starting Wi-Fi STA");
```

```
wifi_prov_mgr_deinit();
wifi_init_sta();
}
xEventGroupWaitBits(wifi_event_group, WIFI_CONNECTED_EVENT, true, true, portMAX_DELAY);

while (1)
{
    for (int i = 0; i < 10; i++)
    {
        ESP_LOGI(TAG, "Hello World!");
        vTaskDelay(1000 / portTICK_PERIOD_MS);
    }

    wifi_prov_mgr_reset_sm_state_for_reprovision();

    xEventGroupWaitBits(wifi_event_group, WIFI_CONNECTED_EVENT, true, true, portMAX_DELAY);
}
}
```

Bước 4: Build project và nạp chương trình vào vi điều khiển, sau đó theo dõi tại màn hình Terminal chúng ta sẽ thấy như sau:



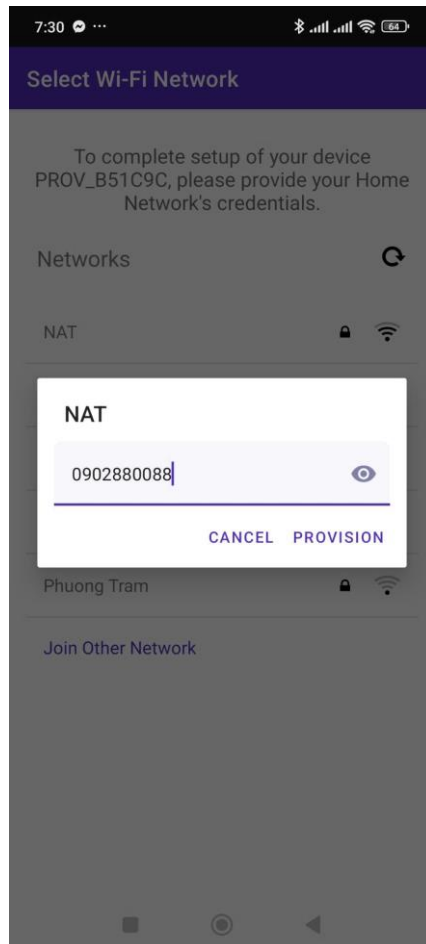

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  MEMORY  XRTOS  ESP-IDF

I (741) QRCODE: Encoding below text with ECC LVL 0 & QR Code Version 10
I (751) QRCODE: {"ver":"v1","name":"PROV_B51C9C","username":"wifiprov","pop":"abcd1234","transport":"ble"}



I (991) app: If QR code is not visible, copy paste the below URL in a browser.
https://espressif.github.io/esp-jumpstart/qrcode.html?data={"ver":"v1","name":"PROV_B51C9C","username":"wifiprov","pop":"abcd1234","transport":"ble"}
```

Bước 5: Sử dụng App **ESP BLE Provisioning** quét mã QR tại màn hình Terminal ESP32 → chọn Điểm truy cập WiFi (Access Point) → nhập Password WIFI → chọn **PROVISION**



PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS MEMORY XRTOS ESP-IDF

```
I (991) app: If QR code is not visible, copy paste the below URL in a browser.
https://espressif.github.io/esp-jumpstart/qrcode.html?data={"ver":"v1","name":"PROV_B51C9C","username":"wifiprov","pop":"abcd1234","transport":"ble"}
I (30681) app: BLE transport: Connected!
I (31451) protocomm_nimble: mtu update event; conn_handle=1 cid=4 mtu=256
I (32741) security2: Using salt and verifier to generate public key...
I (33481) app: Secured session established!
I (53121) NimBLE: GAP procedure initiated: advertise;
I (53121) NimBLE: disc_mode=2
I (53121) NimBLE: adv_channel_map=0 own_addr_type=0 adv_filter_policy=0 adv_itvl_min=256 adv_itvl_max=256
I (53131) NimBLE:

I (53131) app: BLE transport: Disconnected!
I (53141) app: BLE transport: Disconnected!
I (64221) app: BLE transport: Connected!
I (68031) protocomm_nimble: mtu update event; conn_handle=1 cid=4 mtu=256
I (74031) security2: Using salt and verifier to generate public key...
I (75841) app: Secured session established!
W (178561) wifi:Password length matches WPA2 standards, authmode threshold changes from OPEN to WPA2
I (178591) app: Received Wi-Fi credentials
        SSID      : NAT
        Password  : 0902880088
I (184691) wifi:new:<4,0>, old:<1,0>, ap:<255,255>, sta:<4,0>, prof:1, snd_ch_cfg:0x0
I (184691) wifi:state: init -> auth (0xb0)
I (184701) wifi:state: auth -> assoc (0x0)
I (184701) wifi:state: assoc -> run (0x10)
I (184821) wifi:connected with NAT, aid = 5, channel 4, Bw20, bssid = 9c:e9:1c:6f:bc:00
I (184821) wifi:security: WPA2-PSK, phy: bgn, rssi: -60
I (184831) wifi:pm start, type: 1
```